



A dynamic aggregation strategy enhanced efficient global optimization algorithm for solving high-dimensional turbomachinery design problems

Qineng Wang, Zhendong Guo, Yun Chen, Guangjian Ma, Liming Song & Jun Li

To cite this article: Qineng Wang, Zhendong Guo, Yun Chen, Guangjian Ma, Liming Song & Jun Li (2025) A dynamic aggregation strategy enhanced efficient global optimization algorithm for solving high-dimensional turbomachinery design problems, Engineering Optimization, 57:2, 514-542, DOI: [10.1080/0305215X.2024.2325651](https://doi.org/10.1080/0305215X.2024.2325651)

To link to this article: <https://doi.org/10.1080/0305215X.2024.2325651>



Published online: 18 Apr 2024.



Submit your article to this journal [↗](#)



Article views: 263



View related articles [↗](#)



View Crossmark data [↗](#)



Citing articles: 1 View citing articles [↗](#)



A dynamic aggregation strategy enhanced efficient global optimization algorithm for solving high-dimensional turbomachinery design problems

Qineng Wang^a, Zhendong Guo^a, Yun Chen^b, Guangjian Ma^b, Liming Song^a and Jun Li^a

^aInstitute of Turbomachinery, Xi'an Jiaotong University, Xi'an, People's Republic of China; ^bAVIC Shenyang Engine Design Institute, Shenyang, People's Republic of China

ABSTRACT

To address challenges effectively in turbomachinery design optimization involving high-dimensional ($d \geq 30$) expensive black-box problems, a dedicated Efficient Global Optimization (EGO) algorithm is proposed with dynamic aggregation. This specialized approach efficiently navigates optimization tasks with limited sample evaluations. Specifically, the Dynamic Aggregate Efficient Global Optimization (DA-EGO) algorithm decomposes the original high-dimensional design space into low-dimensional subspaces for efficient surrogate-based optimization search, and the optimal solutions of subspaces are combined as an elite-point for the global search. Most importantly, the subspace variables are updated in each iteration, according to the variable interaction analyses in the sub- and full-spaces. The perturbation method and the analysis of variance are used to detect variable interactions. After being validated on 21 benchmark functions ranging from 30 to 90 dimensions, the DA-EGO is used for the optimization of a transonic compressor rotor with 28 variables and a multi-stage compressor optimization with 60 variables. With the above, the effectiveness of the proposed algorithm is well demonstrated.

ARTICLE HISTORY

Received 28 July 2023

Accepted 26 February 2024

KEYWORDS

High-dimensional black-box problem; surrogate-based optimization; knowledge mining

1. Introduction

In the realm of engineering design, the appearance of simulation-based engineering optimization has greatly improved the efficiency and quality of engineering design, so it is widely used in various engineering fields. However, when the engineering problem to be solved is characterized by high dimensionality or is expensive, the difficulty of optimization will be greatly increased (Bouhlel *et al.* 2018; K. Wang, Han, *et al.* 2023; Zhang *et al.* 2020). These two characteristics are ubiquitous in the field of engineering design (Shan and Wang 2010). Taking turbomachinery design optimization as an example (Z. Tang, Xu, and Luo 2022), the high-dimensionality characteristic is reflected in the complex curved shape and large number of design details of turbomachinery components, which require tens or even hundreds of design variables to be described and adjusted accurately. The aerodynamic performance of the turbomachinery components that are designed needs to be obtained through wind tunnel experiments or Computational Fluid Dynamics (CFD) simulations. High-fidelity CFD simulations are very time-consuming, and individual samples can take hours or even days to evaluate. According to different research fields, the specific definitions of high dimensionality and computational expense are also different in the literature. Following the definition in Zhan and Xing (2021),

high dimensionality means that the number of variables varies from 30 to 100, and ‘computationally expensive’ refers to the fact that the samples available for the whole optimization process are limited to thousands.

The optimization process posed a significant computational challenge resulting from the presence of high-dimensional or computationally expensive characteristics (Z. Tang, Xu, and Luo 2022). In tackling high-dimensional problems with dimensions greater than 30, conventional population-based algorithms require millions of samples (Okulewicz, Zaborski, and Mańdziuk 2022). Consequently, such optimization processes often become computationally intensive. Various optimization algorithms developed by researchers have been applied to mitigate this challenge. However, these methods are largely ineffective in solving High-dimension and Expensive Black-box (HEB) problems.

In the optimization of high-dimensional engineering problems, the increase of the number of design variables will lead to an exponential expansion of the design space and a sharp increase in the complexity of the relationship between variables, so that the number of samples needed to complete the optimized search or build the surrogate will also increase exponentially. This difficulty caused by high dimensions is also called the ‘curse of dimensionality’. To solve high-dimensional problems, one strategy is to combine the surrogate model with the population-based algorithm, and use the former to assist and accelerate the search efficiency with so-called Surrogate-Assisted Evolutionary Algorithms (SAEAs) (Li *et al.* 2023; Xie *et al.* 2023). Beyond that, optimization algorithms based on decomposition (also known as divide and conquer algorithms) are the most widely used (Mei *et al.* 2016; Sun, Kirley, and Halgamuge 2018). They decompose the high-dimensional original problem into several smaller and simpler subproblems for optimization. How to decompose the problem is the key to this kind of algorithm. When the decomposition algorithm was put forward, the decomposition scheme was always completely independent of the original problem (Potter and De Jong 1994; Z. Yang, Tang, and Yao 2008), that is, the variables were divided into multiple combinations in a fixed or random way, without considering the characteristics of the problem. Subsequent studies have proved that such a simple ‘manual decomposition’ strategy is unable to deal with problems with complex variable interactions (Meselhi *et al.* 2022; Omidvar *et al.* 2014). Since then, the Variable Interaction-based Decomposition (VID) method has received more widespread attention (Chen *et al.* 2010; Liu *et al.* 2018; Omidvar *et al.* 2014; Omidvar, Li, and Yao 2010). The VID method divides variables into groups according to their interaction relationships between each other. However, it is not suitable for expensive engineering optimization, because it requires numerous samples to obtain the interaction structure of the problem.

In the optimization of expensive engineering problems, the main difficulty is that the increase of single sample cost inevitably leads to the decrease of the total number of available samples. For this reason, researchers developed Surrogate-Based Optimization (SBO) algorithms to solve expensive problems (Forrester, Bressloff, and Keane 2006; Qian and Wu 2008). The SBO method builds an approximate model based on evaluated samples, which significantly reduces the number of samples required by replacing expensive sample evaluation with the explicit surrogate model. A SBO method usually has two key components: the establishment of a surrogate and a search with an acquisitive function. The former summarizes the knowledge in the design space, while the latter uses the knowledge to explore efficiently. However, due to the curse of dimensionality, common surrogate technologies cannot effectively extract the knowledge from the high-dimensional design space, which is also reflected in the decline of the accuracy of the surrogate. As the efficiency of the SBO algorithm largely depends on the accuracy of the established surrogate, in relative terms, the efficiency of optimization for this algorithm is notably higher when the problem dimension is below 15.

In order to solve the HEB problem efficiently, researchers try to combine the surrogate into the decomposition method. The Nash-EGO algorithm (Xu and Chen 2018) directly combines Efficient Global Optimization (EGO) with a manual decomposition strategy. The Self-Evaluation Evolution (SEE) algorithm (P. Yang, Tang, and Yao 2018) uses a novel surrogate in sub-task optimization. In the Cooperative Bayesian Optimization with Hybrid Grouping and Sampling Transfer (CBO-HGST)

algorithm (Jiang, Cheng, and Liu 2022), sub-optimization based on transfer Gaussian process regression are studied. The efficiency improvements of the above three algorithms are only reflected in the subspace, the simple manual decomposition strategy is still used globally. However, algorithms based on the high-dimensional surrogate are not immune to the curse of dimensionality, so they are only applicable to problems with fewer than 30 dimensions. The above algorithms only use the surrogate in a single aspect to reduce costs to a certain extent within the original framework, but they still cannot solve high-dimensional problems efficiently.

In view of the above situation, a new Dynamic Aggregate Efficient Global Optimization (DA-EGO) algorithm is proposed in this article, which is suitable for HEB problems. Similarly, a high-dimensional problem is also decomposed into multiple low-dimensional subproblems by the DA-EGO. The basic idea of the DA-EGO algorithm is to use the surrogate model to conclude the knowledge from the subspaces. Then, this 'local' knowledge is aggregated together into 'global' knowledge of the original design space and used to generate the decomposition scheme in the next cycle. The DA-EGO algorithm proposed in this article is improved in terms of the following aspects in detail.

- (i) Unlike the traditional VID algorithm, which completes the decomposition of space and the construction of subproblems simultaneously, the DA-EGO algorithm has multiple cycles of iteration, and in each cycle, the space decomposition method and the setting of subproblems will gradually change from rough to detailed.
- (ii) In the DA-EGO algorithm, sub-tasks can feed back the decomposition strategy. Knowledge mining after subspace optimization can provide more information about the problem. On the one hand, it can verify the accuracy of existing information, and on the other hand, it can provide new information to help the construction of subsequent sub-tasks.
- (iii) In the DA-EGO, a space reduction mechanism is added, and more knowledge about the design space is used to guide the optimization of subspaces.

The remainder of the article is organized as follows. In Section 2, the related work of the DA-EGO is introduced. Then, the details of the proposed DA-EGO algorithm are illustrated in Section 3. After that, the DA-EGO algorithm is tested on 21 numerical benchmark functions in Section 4. In Section 5, the DA-EGO algorithm is used to optimize a 28-dimensional Rotor 37 transonic compressor cascade, and a typical 60-dimensional multistage compressor. The correctness and effectiveness of the proposed method are well verified. Finally, the conclusions are summarized in Section 6.

2. Related work

2.1. Efficient global optimization (EGO)

The efficient global optimization algorithm is widely used in engineering optimization for its high efficiency (Song *et al.* 2016). The EGO algorithm achieves a great balance between global exploitation and local exploration with two core components, the Kriging surrogate and the Expected Improvement (EI) acquisitive function.

2.1.1. Kriging surrogate

Kriging is a popular surrogate technique (Jones and Schonlau 1998). The Kriging prediction Y_{KG} at unknown site $\mathbf{x}$ is built as a trend function $f(\mathbf{x})$ plus a normal random process $Z(\mathbf{x})$ as

$$Y_{KG}(\mathbf{x}) = f(\mathbf{x}) + Z(\mathbf{x}), \quad (1)$$

where $f(\mathbf{x})$ is usually a constant, linear or quadratic polynomial, and the constant is most widely used; $Z(\mathbf{x})$ describes the local features of Y around the n sample points $X = \{x^{(1)}, \dots, x^{(n)}\}$, which has zero

mean and a co-variance function as

$$\text{cov} \left[Z(\mathbf{x}), Z(\mathbf{x}^{(i)}) \right] = \sigma^2 \exp \left(- \sum_{h=1}^d \theta_h \left\| x_h - x_h^{(i)} \right\|^2 \right). \quad (2)$$

The function prediction and related Mean Squared Error (MSE) at an unknown point $\mathbf{x}$ can be expressed as

$$\begin{aligned} \hat{y}_{\text{KG}}(\mathbf{x}) &= \hat{\mu} + \mathbf{r}^T \mathbf{R}^{-1} (\mathbf{y} - \mathbf{1} \hat{\mu}) \\ s_{\text{KG}}(\mathbf{x}) &= \sigma^2 \{ 1 - \mathbf{r}^T(\mathbf{x}) \mathbf{R}^{-1} \mathbf{r}(\mathbf{x}) \\ &\quad + (1 - \mathbf{1}^T \mathbf{R}^{-1} \mathbf{r}^T(\mathbf{x})) (\mathbf{1}^T \mathbf{R}^{-1} \mathbf{1})^{-1} (1 - \mathbf{1}^T \mathbf{R}^{-1} \mathbf{r}^T(\mathbf{x}))^T \}, \end{aligned} \quad (3)$$

where μ is the regression constant as $\mu = (\mathbf{1}^T \mathbf{R}^{-1} \mathbf{1})^{-1} \mathbf{1}^T \mathbf{R}^{-1} \mathbf{y}$; $\mathbf{R}$ is the correlation matrix $\mathbf{R} := (R(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}))_{i,j} \in \mathbb{R}^{n \times n}$, and $\mathbf{r}$ is the correlation vector $\mathbf{r} := (R(\mathbf{x}^{(i)}, \mathbf{x}))_i \in \mathbb{R}^n$.

2.1.2. Maximum expectation improvement

Once the Kriging surrogate is constructed, the location with maximum EI value can be obtained by using an arbitrary optimizer. The EI function at location $\mathbf{x}$ can be defined as follows:

$$\begin{aligned} \text{EI}(\mathbf{x}) &= (f_{\min} - \hat{y}(\mathbf{x}_{\text{sub},j})) \Phi(u) + s(\mathbf{x}_{\text{sub},j}) \phi(u) \\ u &= (f_{\min} - \hat{y}(\mathbf{x}_{\text{sub},j})) / s(\mathbf{x}_{\text{sub},j}), \end{aligned} \quad (4)$$

where Φ is the normal cumulative distribution function, and ϕ is the normal probability density function.

The conventional SBO algorithms, represented by EGO, are typically limited in their applicability to problems with up to 15-dimensions. Therefore, one of the aims of this study is to extend these sample-efficient methods to higher-dimensional problems.

2.2. Decomposition-based optimization algorithms

In real-world engineering problems, not all variables strongly interact with each other. For example, consider the following six-dimensional problem where interactions occur between x_2, x_3, x_4 and x_5, x_6 .

$$f(\mathbf{x}) := x_1^2 + (x_2 - x_3)^2 + (x_3 - x_4)^2 + (x_5 + x_6)^2. \quad (5)$$

The original problem can be divided into multiple simpler subproblems that do not interact, or weakly interact, with each other:

$$\arg \min_{(x_1, \dots, x_6)} f(x_1, \dots, x_6) = \left(\arg \min_{x_1} f(x_1), \dots, \arg \min_{(x_5, x_6)} f(x_5, x_6) \right). \quad (6)$$

Here, the six variables are divided into three groups, and the optimal location of the original six-dimensional problem can be obtained indirectly by solving three subproblems of lower dimensionality. By utilizing a decomposition strategy, the issue of dimensionality is greatly reduced since the cost of the problem increases linearly with the number of variables rather than exponentially.

As the interaction between subproblems weakens, the optimization solutions of subproblems tend to approach the true optimal solution, but obtaining the relationships among variables requires higher costs. Considering the above trade-off, decomposition strategies can be divided into manual decomposition and variable interaction based decomposition. The manual decomposition approach, which indiscriminately separates variables into multiple groups regardless of their interaction relationships,

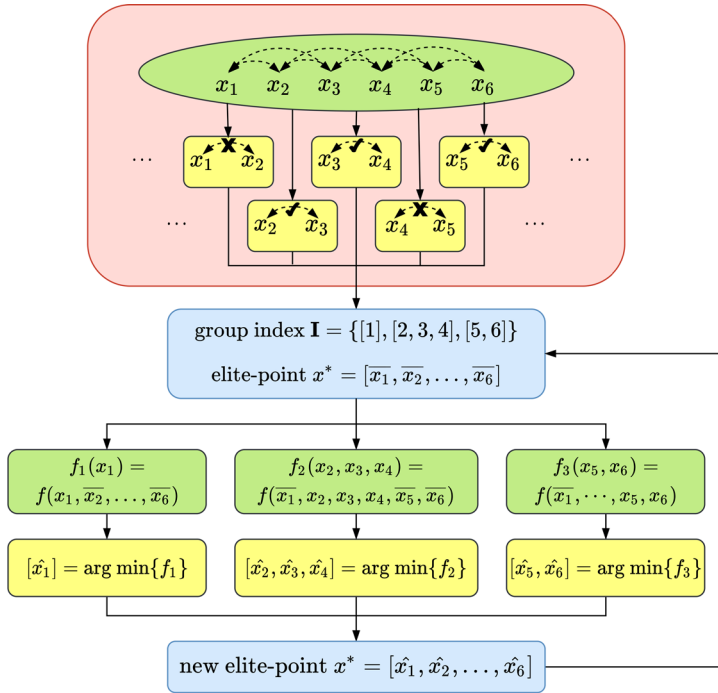


Figure 1. Framework of the variable interaction based decomposition algorithm.

is limited in its effectiveness and is only suitable for fully separable problems. Therefore, this article mainly focuses on the VID method.

VID algorithms are the most popular method for addressing high-dimensional optimization problems. The VID algorithms' vital strategy is to identify variable interactions and leverage the group index $\mathbf{I}$ and elite-point x^* to express specific decomposition arrangements (Z. Yang, Tang, and Yao 2008), the elite-point refers to a point shared by all sub-problems. As shown in Figure 1, the VID algorithm is used on Equation (5). The first step in decomposing a D -dimensional problem is detecting the interaction of each variable pair, which requires $D(D-1)/2$ independent sets of samples. Based on the variable interaction information, the group index $\mathbf{I} = \{I_1, I_2, \dots, I_r\} = \{[x_1], [x_2, x_3, x_4], [x_5, x_6]\}$ is obtained by assigning interacting variables into the same group. Then, the current optimal evaluated samples are selected as the elite-point. Sub-tasks are generated with elite-point x^* and group index $\mathbf{I}$ as Equation (7):

$$\begin{aligned}
 X_{\text{local}} &= [x_1^l, x_2^l, \dots, x_{||I_j||}^l], \quad X_{\text{global}} = [x_1^g, x_2^g, \dots, x_D^g] \\
 f_j^{\text{sub}}(X_{\text{local}}) &= f(G(X_{\text{local}}, I_j, x^*)) = f(X_{\text{global}}) \\
 x_k^g &= \begin{cases} x_k^l & \text{if } x_k \in I_j \\ x_k^* & \text{if } x_k \notin I_j, \end{cases}
 \end{aligned} \tag{7}$$

where, for samples in an $||I_j||$ -dimensional subspace, the values of the remaining $D - ||I_j||$ variables are kept fixed with the elite-point.

Presently, the greatest challenge for implementing VID algorithms on expensive problems is the high cost of sampling for interaction structure detection. The proposed DA-EGO algorithm introduced in this article establishes a new framework that significantly reduces the number of samples needed for variable interaction detection.

2.3. Polynomial chaos expansion based sensitivity analysis

Polynomial Chaos Expansion (PCE) (Sudret 2008; Wiener 1938) is a well-known surrogate technique that represents random variables in terms of polynomial functions, whose prediction function can be expressed as

$$f(x) \approx \mathcal{M}^{(\text{PCE})}(x) = \sum_{\alpha \in \mathcal{A}} a_{\alpha} \psi_{\alpha}(x), \quad \psi_{\alpha}(x) = \prod_{i=1}^D \psi_{\alpha_i}^{(i)}(x_i), \quad (8)$$

where $\psi_{\alpha}(x)$ is a sequence of polynomials, also known as orthogonal polynomial basis functions, and a_{α} is the multi coefficient. The PCE method mainly involves two aspects: the construction of an orthogonal polynomial basis function $\psi_{\alpha}(\cdot)$ and the calculation of the coefficient a_{α} .

First, the Legendre polynomial is selected as the orthogonal basis function $\psi_{\alpha}(\cdot)$, as the PCE will be built with uniformly distributed variables. The Legendre polynomial is expressed in Equation (9), where δ_{ij} is one if $i = j$ and zero if $i \neq j$:

$$\left\langle \psi_i^{(k)}, \psi_j^{(k)} \right\rangle_k = \int_{\mathcal{D}_k} \psi_i(x) \psi_j(x) f_{X_k}(x) dx = \delta_{ij}. \quad (9)$$

Secondly, for the calculation of coefficients, the Orthogonal Matching Pursuit (OMP) algorithm (J. Wang, Kwon, and Shim 2012) is used here to calculate the coefficient a_{α} .

In this study, the PCE surrogates serve as a surrogate model with regression fitting capabilities, especially when working with uniformly distributed random variables. Moreover, an essential feature of PCE is that its coefficients contain critical information about the surrogate model's sensitivity analysis. This information can be exploited to compute global PCE-based Sobol's indices effectively and at significantly lower costs.

Set

$$\mathbf{v} \stackrel{\text{def}}{=} \{i_1, \dots, i_k\} \subset \{1, \dots, M\},$$

and denote by x_v the subvector of x obtained by extracting the components labelled by the indices in v . Extracting all relevant terms of v , form the PCE expression as follows:

$$f_v(x_v) = \sum_{\alpha \in \mathcal{A}_v} \hat{\gamma}_{\alpha} \Psi_{\alpha}(x). \quad (10)$$

The associated sensitivity indices are just the ratio of the above two quantities as shown in Equation (11):

$$\text{Var}[Y_{\mathcal{A}}] = \sum_{\substack{\alpha \in \mathcal{A} \\ \alpha \neq 0}} \hat{\gamma}_{\alpha}^2, \quad \text{Var}[f_v(x_v)] = \sum_{\substack{\alpha \in \mathcal{A}_v \\ \alpha \neq 0}} \hat{\gamma}_{\alpha}^2. \quad (11)$$

For more details on PCE, the interested reader may refer to Xiu (2010). The construction of basis functions, the coefficient calculation, and the sensitivity analysis of the PCE model involved in this article are all based on UQLab (Marelli and Sudret 2014), an open source library of MATLAB[®].

3. The proposed method

As mentioned in the introduction, the DA-EGO algorithm is proposed to solve the many existing HEB problems in the field of aerodynamic optimization design. Figure 1 shows the overall framework of the DA-EGO algorithm, which can be divided mainly into three parts: (i) initialization; (ii) a decomposition strategy in high-dimensional space; and (iii) subproblem optimization and knowledge mining. After the initialization of the problem is completed, the other two parts will proceed in turn until the optimization stop condition is met. The DA-EGO algorithm establishes an effective loop

for ‘constructing sub-tasks based on knowledge of the design space’ and ‘enriching knowledge of the design space through data mining of sub-tasks’ in high-dimensional problems, where knowledge of the design space contains three types of data: sampling, variable interactions and search boundaries.

An important reason for the DA-EGO being able to solve HEB problems efficiently is that it breaks the barriers of knowledge transfer in the original high-dimensional space and subproblem space, so that the knowledge obtained from the low-dimensional space can be applied to the high-dimensional space. In the following sections, the DA-EGO algorithm will be introduced in detail.

3.1. Initialization

In the initialization process, the three kinds of data contained in the knowledge of the problem need to be initialized in turn. For the initial sampling, the algorithm uses the Latin Hypercube Sampling (LHS) method to sample 50 initial samples uniformly in the design space and to evaluate them. In addition, at the beginning of the optimization, there is no interaction between any variables by default, and the search scope is the entire design space, which means the search range for each variable is $[0, 1]$ in the beginning.

3.2. High-dimensional problem decomposition

In the problem decomposition part, all the knowledge information about the design space will be collected and summarized, and then the new sub-optimization problems in the new cycle will be constructed based on this knowledge. Figure 2 shows the data flow diagram of the DA-EGO algorithm. The knowledge reserve of the three kinds of data are represented respectively by the three tables at the top of the figure. These tables record the following: (i) the coordinates and values of all evaluated samples; (ii) the interacting variable pairs found; (iii) the current search range corresponding to each variable. After receiving the knowledge provided from the subproblem knowledge mining, the knowledge of the original problem needs to be updated, including the update of the elite-point, decomposition scheme and search scope.

As described in Section 2.1, the elite-point is a crucial part of decomposing a high-dimensional problem. Here, the current best sample among all the evaluated samples will be selected as the new elite-point. The updates to the decomposition scheme and search scope are more complex, and will be described in detail below.

3.2.1. Decomposition scheme updating

The strategy for decomposing high-dimensional problems relies on identifying interactions among variables. In the absence of precise interaction coefficients between $D(D-1)/2$ variable pairs, the DA-EGO algorithm classifies relationships into three categories: (i) confirmed interactions; (ii) suspected interactions; and (iii) non-interactions (here used as a term to describe variable relationships, possibly indicating weak interactions). The variable pairs determining interactions are denoted as $P_C = (x_1^{c1} x_1^{c2}, x_2^{c1} x_2^{c2}, \dots, x_t^{c1} x_t^{c2})$, which are documented in the first table in Figure 3 and contain t variable pairs. Suspected interactions are identified from the subset of sample pairs with the highest sensitivity coefficient $S_{ij}^{PCE} (1 \leq i < j \leq D)$ among the remaining $D(D-1)/2 - t$ sample pairs, which are denoted as $P_S = (x_1^{s1} x_1^{s2}, x_2^{s1} x_2^{s2}, \dots, x_{D(D-1)/2}^{s1} x_{D(D-1)/2}^{s2})$, where D represents the dimensions of the original problem. The dotted box on the left side Figure 3 is the set of variable pairs P_S . It is worth noting that, due to the existence of the curse of dimensionality, the accuracy of the high-dimensional PCE surrogate is limited, which will lead to errors in the interactive variable pair selection. After the P_C and P_S are obtained, the decomposition of the original problem is completed based on the following steps to obtain the final variable decomposition scheme. The pseudo-code of the decomposition scheme update part is shown in Algorithm 1.

Furthermore, suspected interactions are confirmed through validation by low-dimensional surrogate models. The details of this confirmation method are explained in Section 3.2.2.

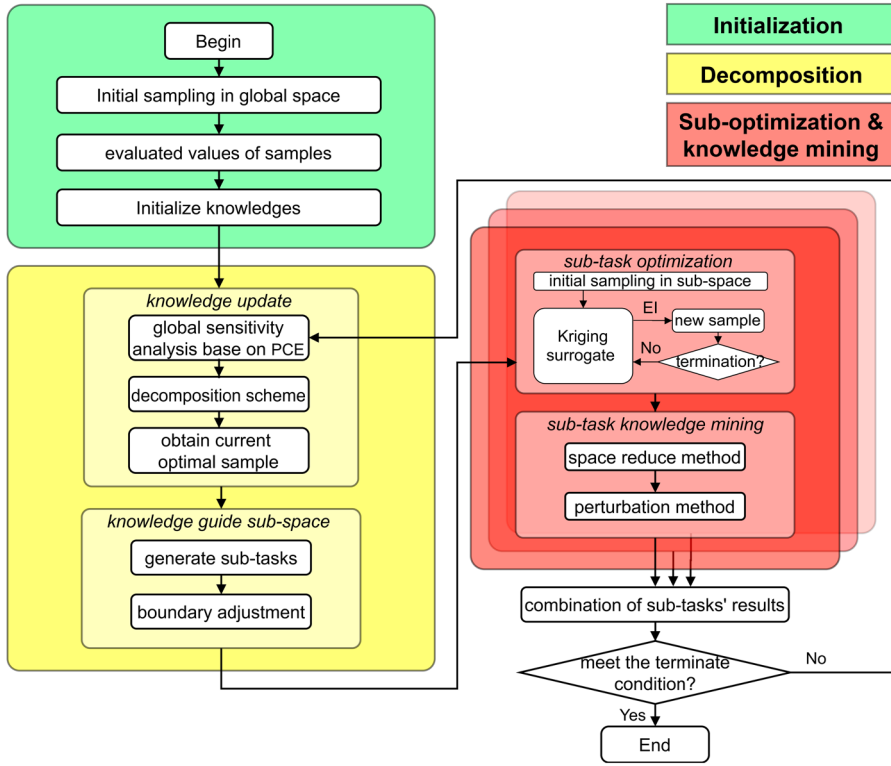


Figure 2. Framework of the proposed DA-EGO algorithm.

3.2.2. Updating the search scope

The update of the search scope is associated with the decomposition scheme. Based on knowledge of the design space, the search range of the subproblems can be reduced, which can significantly enhance optimization efficiency and knowledge mining accuracy. However, the adjustment of the search range is based on accumulated experience. If similar sub-tasks have been previously solved (meaning subproblems with the same variables, but different elite-points), the new search can be based on previous experience. Conversely, if a new sub-task is unknown (meaning new variables are added to the group), the new problem must be thoroughly explored. Equation (12) shows the specific method:

$$S_j = \begin{cases} [l_k, u_k]^{\|I_j\|} & \text{if the variable in } I_j \text{ remains the same} \\ [0, 1]^{\|I_j\|} & \text{if the variable in } I_j \text{ changes} \end{cases} \quad (12)$$

where l_k and u_k are the lower and upper boundary values of the k th variable in the j th subset design space. These variable ranges are obtained in previous cycles of sub-tasks and recorded in the boundary table shown in Figure 3.

When the elite-point, decomposition scheme and search scope are all updated, new sub-tasks are created based on this knowledge. During the optimization of the samples in the subspace, only the values of the variables corresponding to the subspace change, while the remaining variables retain the values of the elite-point. Following these steps, subproblem construction for the new optimization cycle is complete. A thorough comprehension of the original problem leads to strong interactions within the subproblems but weak interactions between them, resulting in a search boundary effectively narrowed down to the optimal region. This procedure fully illustrates how design space knowledge can be used to construct sub-optimization problems effectively.

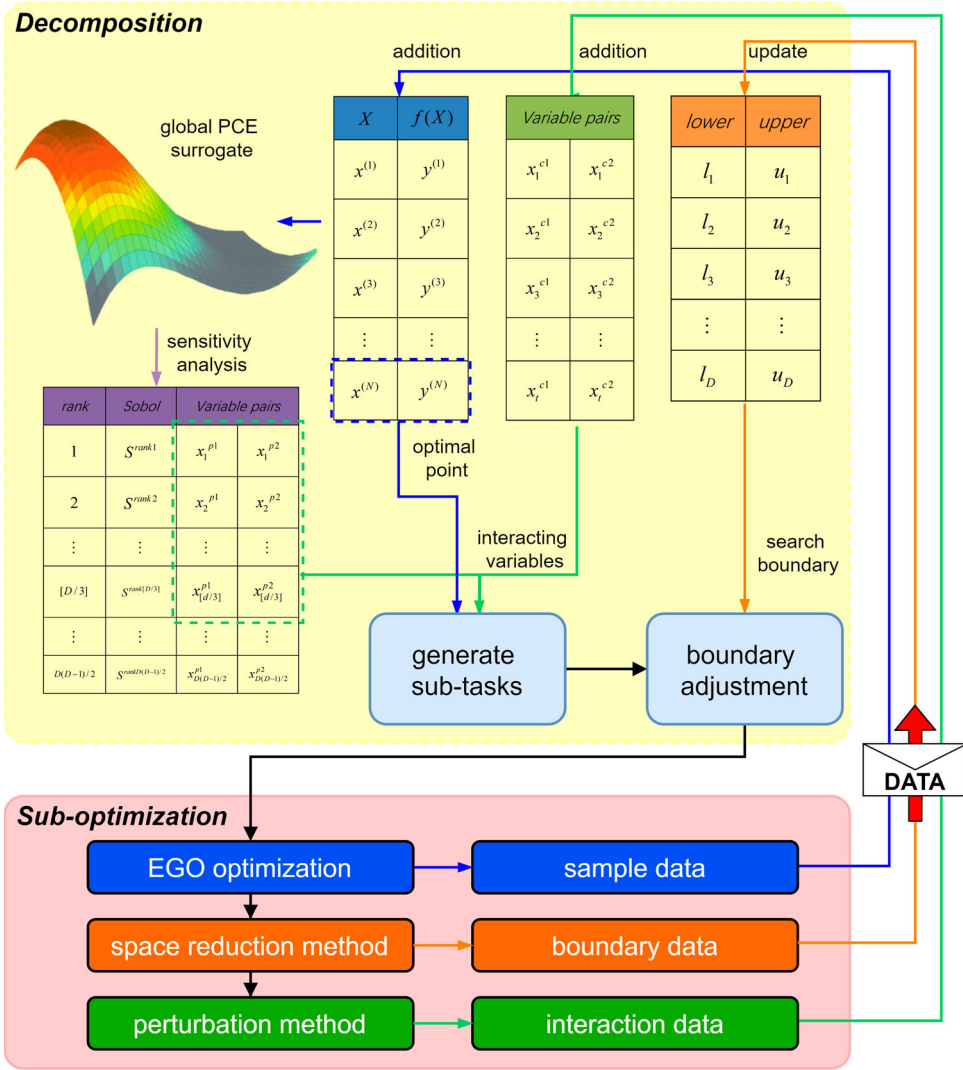


Figure 3. The data flow of the proposed DA-EGO algorithm.

3.3. Sub-task optimization and knowledge mining

The sub-task optimization and knowledge mining module is faced with an independent sub-task as shown in Equation (13), which contains only several variables of the original problem, so its dimension is low:

$$\begin{aligned} \min \quad & f_j^{\text{sub}}(\mathbf{x}) = \min f(G(\mathbf{x}, I_j, \mathbf{x}^*)) \\ \text{s.t.} \quad & l_k \leq \mathbf{x}_k \leq u_k \quad (\mathbf{x}_k \in I_j), \end{aligned} \quad (13)$$

where the function $G(\cdot)$ denotes the mapping relationship between subspace and global space, which has been defined in Equation (7). The design space of subproblems is totally determined by the elite-point $\mathbf{x}^*$, the decomposition scheme (which is expressed as group index I) and the search scope $[L, U]$ together.

Algorithm 1 : Updating of the decomposition scheme**Require:**

the dataset of sorted variable pairs (P_S); the dataset of accumulated interacting variable pairs (P_C);

Ensure:

the final decomposition scheme (**I**);

```

1: for  $i = 1 \rightarrow D$  do
2:    $I_i \leftarrow x_i$ 
3: end for
4: for  $k = 1 \rightarrow t$  do
5:    $[x_k^{c1}, x_k^{c2}] \leftarrow P_{C,k}$ 
6:    $\exists x_k^{c1} \in I_{c1} \quad (c1 \in [1, D])$  // find the group contain  $x_k^{c1}$ 
7:    $\exists x_k^{c2} \in I_{c2} \quad (c2 \in [1, D])$ 
8:    $I_{c1} \leftarrow I_{c1} \cup I_{c2}$ 
9:    $I_{c2} \leftarrow \emptyset$ 
10: end for
11:  $k \leftarrow 1$ 
12: while  $k \leq [D/3]$  do
13:    $[x_k^{s1}, x_k^{s2}] \leftarrow P_{S,k}$ 
14:    $\exists x_k^{s1} \in I_{s1} \quad (s1 \in [1, D])$ 
15:    $\exists x_k^{s2} \in I_{s2} \quad (s2 \in [1, D])$ 
16:   if  $|I_{s2} \cup I_{s2}| \leq \Delta$  and  $[x_k^{s1}, x_k^{s2}] \notin P_C$  then
17:      $I_{s1} \leftarrow I_{s2} \cup I_{s2}$ 
18:      $I_{s2} \leftarrow \emptyset$ 
19:      $k \leftarrow k + 1$ 
20:   end if
21: end while
22: return I

```

3.3.1. Sub-task optimization

First, the EGO algorithm is used to search for the optimal sample in the subproblem space. As described in Section 2.3, the EGO algorithm usually uses the maximum Expected Improvement (EI) criterion to achieve a balance between local search and global exploration. Therefore, the accuracy of the Kriging surrogate established will be continuously improved with new samples being added. In this article, the initial sampling number of the EGO algorithm is set to $3d$, the maximum number of iterations is set to $6d$, and here d is the subspace dimension. After the optimization, the optimal sample and the predictive surrogate $\hat{f}_{\text{sub}}^{\text{KG}}$ of the subspace can be obtained.

3.3.2. Sub-task knowledge mining

After obtaining the optimal sample of the subproblem, the search scope and variable interaction of the subproblem need to be obtained through knowledge mining technology. The search boundary of the subproblem can be redefined based on the obtained surrogate $\hat{f}_{\text{sub}}^{\text{KG}}$. In the DA-EGO algorithm, the space reduction method is used to learn the new boundary in the subproblem. The space reduction method can make the optimization search focus on the potential optimal region and avoid unnecessary calculation cost in the non-important region. The detailed process of the space reduction is expressed as pseudo-code in Algorithm 2. In this article, the sampling number N_{SR} in the surrogate $\hat{f}_{\text{sub}}^{\text{KG}}$ is set to 10,000, and the space reducing ratio ω is set to 0.5.

In the DA-EGO algorithm, the common interaction analysis method perturbation method is used to analyse the interaction between variables in the same sub-tasks. The perturbation method evaluates variable relationships one pair at a time. Consequently, a problem with d variables requires

Algorithm 2 : Space reduction in the j th sub-task**Require:**

the sampling number(N_{SR}); the surrogate $\hat{f}_{\text{sub}}^{\text{KG}}$; the current searching range of the j th sub-task ($[l_k, u_k] \ (x_k \in I_j)$); the space reducing ratio (ω);

Ensure:

the new searching range of the j th sub-task ($[l_k, u_k] \ (x_k \in I_j)$);

- 1: $[X^1, \dots, X^{N_{\text{SR}}}] \leftarrow$ generate N_{SR} samples in the range $[l_k, u_k] \ (x_k \in I_j)$
- 2: **for** $k = 1 \rightarrow N$ **do**
- 3: $\hat{y}^k \leftarrow \hat{f}_{\text{sub}}^{\text{KG}}(X^k)$
- 4: **end for**
- 5: $[X^{\text{rank}1}, \dots, X^{\text{rank}N_{\text{SR}}}] \leftarrow$ sort $[X^1, \dots, X^{N_{\text{SR}}}]$ in ascending order according to $\hat{y}$
- 6: $M \leftarrow \lfloor \omega \cdot N_{\text{SR}} \rfloor$
- 7: **for** $k = 1 \rightarrow D$ **do**
- 8: $l_k \leftarrow \min(x_k^{\text{rank}1}, x_k^{\text{rank}2}, \dots, x_k^{\text{rank}M})$
- 9: $u_k \leftarrow \max(x_k^{\text{rank}1}, x_k^{\text{rank}2}, \dots, x_k^{\text{rank}M})$
- 10: **end for**
- 11: **return** $[l_k, u_k] \ (x_k \in I_j)$

the evaluation of $d(d-1)/2$ variable pairs separately. Moreover, the perturbation analysis methodology, based on a surrogate, can improve the accuracy of results by removing regions of the surrogate model with low accuracy. According to the literature (Kang and Lee 2020), limiting the use of the perturbation method to the range $[0.1, 0.9]^D$ significantly enhances its precision. Thus, restricting the perturbation analysis sampling within a reasonable range can improve its accuracy. According to the established subspace Kriging model $\hat{f}_{\text{sub}}^{\text{KG}}$, the perturbation method is used to confirm whether the selected variable pairs have interactive relations. Equation (14) is the specific method of perturbation analysis, in which parameter values are consistent with those in the literature (Kang and Lee 2020), set $N_{\text{PA}} = 10,000$, and $h = 0.0001$:

$$\begin{aligned}
 H_{ij} = & \frac{1}{N_{\text{PA}} h^2 (\max(\hat{f}_{\text{sub}}^{\text{KG}}) - \min(\hat{f}_{\text{sub}}^{\text{KG}}))} \sum_{k=1}^{N_{\text{PA}}} \left| \hat{f}_{\text{sub}}^{\text{KG}} \left(x_1^k, x_2^k, \dots, x_i^k + h, \dots, x_j^k + h, \dots, x_d^k \right) \right. \\
 & - \hat{f}_{\text{sub}}^{\text{KG}} \left(x_1^k, x_2^k, \dots, x_i^k + h, \dots, x_j^k, \dots, x_d^k \right) \\
 & - \hat{f}_{\text{sub}}^{\text{KG}} \left(x_1^k, x_2^k, \dots, x_i^k, \dots, x_j^k + h, \dots, x_d^k \right) \\
 & \left. + \hat{f}_{\text{sub}}^{\text{KG}} \left(x_1^k, x_2^k, \dots, x_i^k, \dots, x_j^k, \dots, x_d^k \right) \right|. \quad (14)
 \end{aligned}$$

If the condition $H_{ij} \geq 0.2$ is met, the interaction between variable x_i and x_j is confirmed. Then, these confirmed interacting variable pairs are recorded and fed back to the problem decomposition part.

With the completion of the above steps, all evaluation samples of the subproblem, new search boundaries, and interacted pairs of variables can be obtained. As shown in Figure 3, this knowledge will be extracted from each sub-task and concentrated into the three tables storing the original problem information: (i) the sample points of the subspace will be added to the evaluated data table; (ii) the variable pairs that actually have interactive relations will be added to the interactive table; and (iii) the obtained search boundary will replace the original boundary in the search boundary table. After obtaining these feedbacks, the knowledge of the global problem becomes more accurate and rich, which is also conducive to the construction of the following subproblems.

As knowledge is received in each cycle, problems are decomposed and subproblems optimized repeatedly while gaining increasingly better samples, and interaction structures of original high-dimensional problems become clearer. This gradual exploration also enables smaller search scopes, which are ideal for enhancing local search capabilities.

4. Numerical experimental study

In this section, the performance of the proposed DA-EGO algorithm is demonstrated on 21 benchmark functions, and the DA-EGO algorithm is also compared with state-of-the-art algorithms.

4.1. Experimental settings

To show the effectiveness of the proposed algorithm, it is compared against the following three kinds of algorithm.

- (1) The first kind are the model-free algorithms including the classical Genetic Algorithm (GA) (Mitchell 1998), Differential Evolution (DE) algorithm (Storn and Price 1997) and a recently proposed Gaining–Sharing Knowledge (GSK) based algorithm (Mohamed, Hadi, and Mohamed 2020). For these 3 algorithms, their population size is set to be 50 and the remaining variables are consistent with the default values shown in the related articles.
- (2) The second kind are the surrogate assisted genetic algorithms including the Incremental Kriging-Assisted Evolutionary Algorithm (IKAEA) (Zhan and Xing 2021) and the Generalized Surrogate-assisted Genetic Algorithm (GSGA) (Cai *et al.* 2019). Note that the IKAEA and GSGA algorithms are recently proposed SAEAs for high-dimensional optimization problems. The default settings in the articles on the IKAEA and GSGA algorithms are used in the following tests.
- (3) The third kind is the surrogate-based algorithm, Nash-EGO, which has been mentioned in the introduction. Additionally, the Random Grouping Efficient Global Optimization (RG-EGO) algorithm, a variant of the DA-EGO algorithm, is also joined in the comparison. The RG-EGO used a random grouping decomposition strategy while keeping all other settings the same as for the DA-EGO algorithm. Therefore, comparing the performance of the two can clearly reflect the effect of the novel interaction analysis strategy proposed in this article.

One of the principles for the selection of comparison algorithms is the consistency of the applicable dimensions and the range of sample sizes. So, some algorithms with similar mechanisms but different scopes of application—see P. Yang, Tang, and Yao (2018) and OMID (Hajikolaie and Wang 2014)—are not involved in the comparison.

In this article, the tests for all the benchmark problems are repeated 20 times. In the i th test of each benchmark function ($i = 1, \dots, 20$), the initial sample distribution (called the initial population) of all the algorithms is the same. The termination condition is when the Number of Function Evaluations (NFEs) reaches 1500.

4.2. Benchmark functions

All the benchmark functions used in this article were selected from the CEC's 2010 Special Session (K. Tang *et al.* 2009). The characteristics of these benchmark functions are summarized in Table 1 and their formulations are shown in Table 2.

Note that the variable interactions have a great impact on the performance of the optimization algorithm. Therefore, nine separable and nine non-separable function are selected for the test. Specifically, the separable functions are the functions whose design variables are independent. In contrast,

Table 1. Benchmark functions.

Index	Dim	Name	Property	Range
F1	30/60/90	Shifted elliptic	Separable	$[-10, 10]^D$
F2	30/60/90	Shifted Rastrigin	Separable	$[-5, 5]^D$
F3	30/60/90	Shifted Ackley	Separable	$[-32, 32]^D$
F4	30/60/90	Rotated elliptic	Partially-separable	$[-10, 10]^D$
F5	30/60/90	Rotated Rastrigin	Partially-separable	$[-5, 5]^D$
F6	30/60/90	Rotated Ackley	Partially-separable	$[-32, 32]^D$
F7	30/60/90	Shifted Rosenbrock	Non-separable	$[-10, 10]^D$

Table 2. Equations of benchmark functions.

Index	Description
F1	$f(x) = F_{\text{elliptic}}(Z_{\text{sft}}(x))$
F2	$f(x) = F_{\text{Rastrigin}}(Z_{\text{sft}}(x))$
F3	$f(x) = F_{\text{Ackley}}(Z_{\text{sft}}(x))$
F4	$f(x) = F_{\text{elliptic}}(Z_{\text{rot}}(x))$
F5	$f(x) = F_{\text{Rastrigin}}(Z_{\text{rot}}(x))$
F6	$f(x) = F_{\text{Ackley}}(Z_{\text{rot}}(x))$
F7	$f(x) = F_{\text{Rosenbrock}}(Z_{\text{sft}}(x))$
F_{elliptic}	$f(z) = \sum_{i=1}^D (10^6)^{(i-1)/(D-1)} z_i^2$
$F_{\text{Rastrigin}}$	$f(z) = \sum_{i=1}^D [z_i^2 - 10 \cos(2\pi z_i) + 10]$
F_{Ackley}	$f(z) = -20 \exp \left(-0.2 \sqrt{\frac{1}{D} \sum_{i=1}^D z_i^2} \right) - \exp \left(\frac{1}{D} \sum_{i=1}^D \cos(2\pi z_i) \right) + 20 + e$
$F_{\text{Rosenbrock}}$	$f(z) = \sum_{i=1}^{D-1} [100(z_i^2 - z_{i+1})^2 + (z_i - 1)^2]$
Z_{sft}	$f(x) = (x - O)[P_1 : P_D]$
Z_{rot}	$\begin{cases} f(x)[1 + 5n : 5 + 5n] = Z_{\text{sft}}(x)[1 + 5n : 5 + 5n] \times M & (\text{if } n = 0, \dots, d/10 - 1) \\ f(x)[1 + 5n : 5 + 5n] = Z_{\text{sft}}(x)[1 + 5n : 5 + 5n] & (\text{if } n = d/10, \dots, d/5) \end{cases}$

the design variables of the non-separable functions are highly interacting. More specifically, the operations in K. Tang *et al.* (2009), such as shifting, rearrangement and rotation to build the benchmark functions, are used as shown in Table 2. In particular, the expressions of O , M and P can be found in previous work (Q. Wang, Song, *et al.* 2023).

4.3. Results of benchmark functions

4.3.1. Comparison of results on 30-D benchmark functions

The convergence curves of 6 different 30-dimensional benchmark functions are given in Figure 4. The details of the optimization results are listed in Table A1 of the appendix.

As shown in Table A1 in the appendix, the DA-EGO algorithm performed significantly better than all the other seven compared algorithms in all 30-D test cases. The values of standard deviation reflect the robustness of the algorithm for the initial distribution.

Here, benchmark functions are discussed separately according to the separability. In Figures 4(a), 4(d) and 4(g), benchmark functions F1, F2 and F3 are separable functions. Those decomposition based algorithms such as Nash-EGO, DA-EGO and RG-EGO have advantages, as the combination of all the sub-optimization results is bound to get a better solution for separable functions. However, the GSGA algorithm also has great performance on the multi-modal functions F2 and F3. Comparing the DA-EGO and RG-EGO algorithms in detail, it is found that both of them have

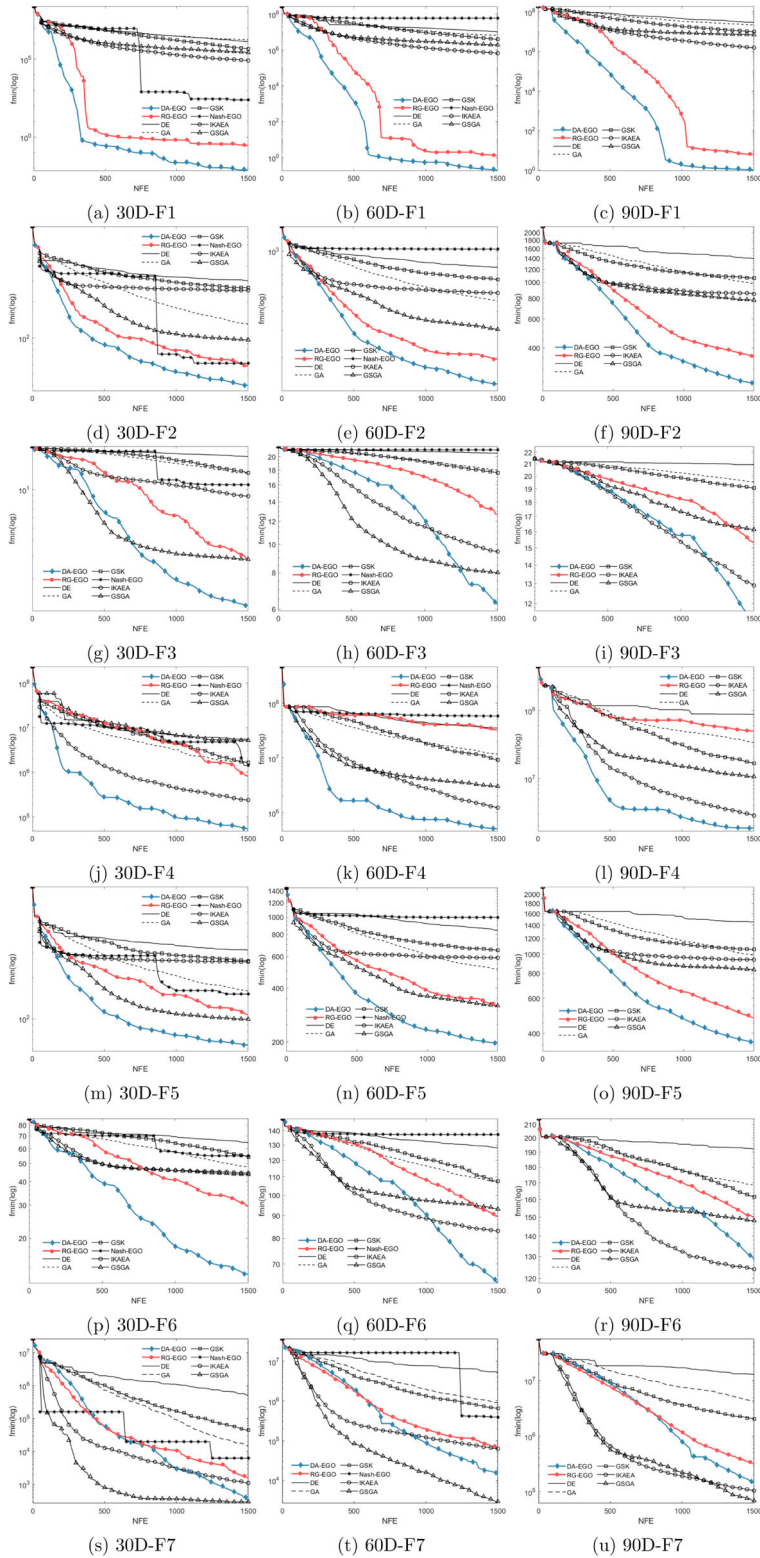


Figure 4. Average convergence curves of 7 algorithms on 21 benchmark functions. (a) 30D-F1. (b) 60D-F1. (c) 90D-F1. (d) 30D-F2. (e) 60D-F2. (f) 90D-F2. (g) 30D-F3. (h) 60D-F3. (i) 90D-F3. (j) 30D-F4. (k) 60D-F4. (l) 90D-F4. (m) 30D-F5. (n) 60D-F5. (o) 90D-F5. (p) 30D-F6. (q) 60D-F6. (r) 90D-F6. (s) 30D-F7. (t) 60D-F7 and (u) 90D-F7.

similar convergence curves, but DA-EGO has higher efficiency owing to the existence of its space reduction strategy.

On the other hand, in Figures 4(j), 4(m) and 4(p), benchmark functions F4, F5 and F6 are partially-separable functions, *i.e.* half their variables strongly interact with each other. For such problems, the key to the efficient search of the decomposition based algorithm is whether the problem can be decomposed accurately according to the interaction. Two algorithms using a manual decomposition strategy, Nash-EGO and RG-EGO, lose their advantages compared with other algorithms. In Figure 4(s), benchmark function F7 is a non-separable function and all variables strongly interact with each other. In this case, the initial convergence speed of DA-EGO is a bit slower, but the optimal solution searched after 1500 evaluations is basically consistent with the GSDE and IKAEA algorithms. In contrast, the convergence rate and the final solutions of DA-EGO are still significantly better than the compared algorithms.

4.3.2. Comparison of results on 60-D benchmark functions

The convergence curves of 6 different 60-dimensional benchmark functions are given in Figure 4. The details of the optimization results are listed in Table A2 of the appendix.

As shown in Table A2, the DA-EGO algorithm performs significantly better than all the other seven compared algorithms in all 60-D test cases. Similar to the results of the test of the 30-dimensional cases, both DA-EGO and RG-EGO have clear advantages in the test on the F1 function. However, in the test on the F4 function, a variant of F1 with strong interactions, the DA-EGO still has high efficiency but the RG-EGO is difficult to converge. The above huge differences between the two cases shows the role of the proposed screening and identification strategy in the face of strong interaction problems. The DA-EGO algorithm accurately decomposes the original problems with the interaction information obtained, thus different strategies can be adopted towards the fully separable problems or the problems with strong interactions. On the contrary, the random grouping strategy treats all problems in the same way, and its performance becomes worse with increasing number of dimensions. Another noteworthy phenomenon is that the convergence rate of DA-EGO is slower than that of GSGA or IKAEA in the early stages and is much faster in the latter stages. That's because the DA-EGO algorithm works based on the interaction analysis result, which is much more complex in problems with higher dimension and need more accumulation of evaluated samples. However, in the latter stages, the above obtained interaction information help the DA-EGO algorithm keep a fast convergence rate.

Table A3 in the appendix presents a comparison of the average time taken by various algorithms for calculating F1 functions across different dimensions. It is evident that the population-based algorithm exhibits significantly shorter calculation times. In the agent model-based approach, IKAEA employs the enhanced Kriging model to reduce calculation times markedly. The computation times for DA-EGO, GSGA and Nash-EGO methods are comparable. Given that this article focuses on expensive problems, the evaluation time for sample assessments surpasses the computing resource consumption during algorithm execution. Therefore, reducing the computational complexity is not the primary research goal. For addressing expensive problems, enhancing an algorithm's parallel capability often proves more effective than reducing its computational complexity. In this context, a decomposition-class algorithm shares the same advantage as a population-based algorithm.

4.3.3. Comparison of results on 90-D benchmark functions

The convergence curves of 6 different 90-dimensional benchmark functions are given in Figure 4. The details of the optimization results are listed in Table A4 of the appendix. The test results demonstrated that the DA-EGO algorithm has the ability to solve 90-D problems efficiently.

The test results on the above 21 functions from 30-D to 90-D fully reflect the advantages of the DA-EGO algorithm over the other compared algorithms. The proposed algorithm achieved the best overall results because it completes the interactive analysis under the premise of high-dimensionality and expensiveness, and decomposes the original problem accurately. The efficiency advantage of

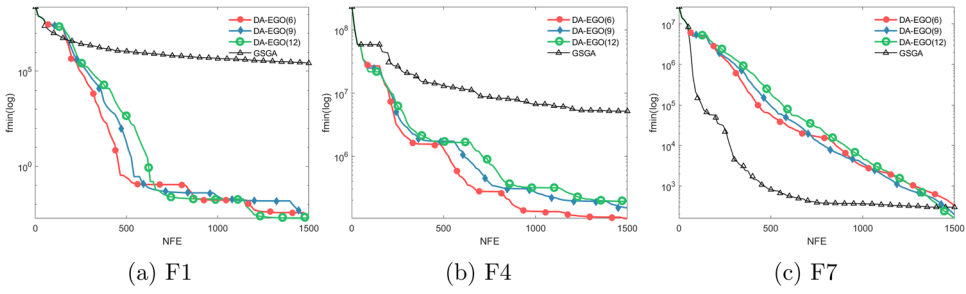


Figure 5. Average convergence curves of the DA-EGO algorithm with different iteration numbers of a sub-optimization. (a) F1. (b) F4 and (c) F7.

identifying the interactive structure of the problem makes DA-EGO maintain a high convergence ratio even in the latter stages of optimization. The above test results show that DA-EGO can achieve the purpose of solving the HEB problem efficiently.

4.4. Parameter sensitivity analysis

In the DA-EGO algorithm, $iter_{\max}$ is a crucial parameter that indicates the maximum iteration time in each sub-task. For divide-and-conquer based algorithms, the target of each sub-task is to achieve complete convergence in previous work, as the convergent optimal solution is the only significant output. So, sub-tasks have sufficient computing resources, *i.e.* the number of evaluated samples is 24 times the subspace's dimension in Nash-EGO.

However, in the DA-EGO algorithm, the dynamic aggregated mechanism broadens the selectable range of $iter_{\max}$. Here, define $iter_{\max}$ as linearly related to d : $iter_{\max} = t_{\text{iter}} \cdot d$. In order to explore the parameter sensitivity of $iter_{\max}$, three different t_{iter} values, [6, 9, 12], are adopted in the test. Figure 5 shows the average convergence curve of the DA-EGO algorithm on benchmark function F1, where the numbers in parentheses are the corresponding t_{iter} values. The convergence curves of the GSGA algorithm are also drawn for comparison. It can be seen from the figure that changes in t_{iter} have little impact on the overall convergence efficiency of the DA-EGO algorithm. Using a small number of iterations in a certain range is conducive to speeding up the whole dynamic decomposition process, so that the algorithm can obtain the interactive information of the problem faster and accelerate the convergence.

5. Engineering test cases

As mentioned in the introduction, the design of turbomachinery components are typical HEB problems, especially the joint design of multiple turbomachinery components. After using benchmark functions to prove the efficiency and robustness of the DA-EGO algorithm, in this section, two engineering optimization design tasks will be solved by the proposed algorithm.

The compressor is a turbomachine that converts mechanical energy into fluid kinetic energy and potential energy. As an important part of a gas turbine, the performance of the compressor has a direct impact on the efficiency, power and reliability of the gas turbine. Optimization of the design of the compressor is a difficult task. Owing to the strong three-dimensional effect of the flow in the compressor, the shape of the compressor blade is usually very complex, and the profile shape from the root to the tip varies greatly, with obvious bending and sweeping at the same time. Therefore, more design variables are needed to shape the compressor blade accurately.

Moreover, in a multistage axial flow compressor, each row of blades is disturbed by upstream and downstream blades. So matching between different blades has a great impact on the efficiency of the compressor, and multistage blades should to be considered as a whole in the design. However,

Table 3. Design variables for the engineering test case.

Geometric definition	Variable index
Control coefficient in 0% span	$x_1, \dots, x_5$
Control coefficient in 25% span	$x_6, \dots, x_{10}$
Control coefficient in 50% span	$x_{11}, \dots, x_{15}$
Control coefficient in 75% span	$x_{16}, \dots, x_{20}$
Control coefficient in 100% span	$x_{21}, \dots, x_{25}$
Circumferential translation parameter	x_{26}
Axial translation parameter	x_{27}, x_{28}

multistage compressor design brings two problems: (i) expansion of the calculation domain leads to an increase in the number of grids, and usually the computational cost of each performance evaluation increases linearly with the number of blades; and (ii) an increase in the number of blades leads to an increase in the number of design variables. Usually, the scale of problem-solving increases exponentially with the number of design variables.

In this section, a single compressor design task with 28 design variables and a multistage compressor design task with 60 variables are completed with the proposed DA-EGO algorithm, and the DA-EGO's performance is compared with another four algorithms, with the same initial distribution of 50 samples as that generated by the MATLAB built-in function 'lhsdesign'.

5.1. Engineering test case 1: rotor 37 blade

In the first engineering test case, the proposed DA-EGO optimization algorithm is applied to the aerodynamic optimization of the well-known Rotor 37 blade (Suder 1996).

5.1.1. Problem description

Here, five section profiles of 0%, 25%, 50%, 75% and 100% span are selected, and five active control points on the suction side of each section are selected to adjust the section profiles. At the same time, the bending and sweeping of the blade are also adjusted. The number of design variables in the optimization of Rotor 37 is 28; these variables are shown in Table 3. Figure 6 shows the generation of three-dimensional blade geometric modelling. The parameters of the five control points of each section determine the shape of this section. After finishing the parameterization of all section profiles, one circumferential translation parameter x_{26} for the middle section, and two axial translation parameters x_{27}, x_{28} for the tip section and middle section, are selected to adjust the stacking line in 3-D space. Then, a 3-D blade profile is obtained using skinning surface techniques. The isentropic efficiency η_{is} is set to be the objective function for the optimization, the related optimization model is shown below:

$$\begin{aligned}
 \max \quad & \{f_{obj}(\mathbf{x})\} = \max\{\eta_{is}(\mathbf{x})/p(\mathbf{x})\} \\
 \text{s.t.} \quad & 0.98 \cdot m(\text{ref}) \leq m(\mathbf{x}) \leq 1.02 \cdot m(\text{ref}) \\
 & 0.98 \cdot Pr(\text{ref}) \leq Pr(\mathbf{x}) \leq 1.02 \cdot Pr(\text{ref}) \\
 \text{where} \quad & \eta_{is} = \left\{ \left(p_{\text{outlet}}^T / p_{\text{inlet}}^T \right)^{(\gamma-1)/\gamma} - 1 \right\} / \left(T_{\text{outlet}}^T / T_{\text{inlet}}^T - 1 \right)
 \end{aligned} \tag{15}$$

where 'ref' means the reference design, m is the mass flow rate, $Pr = p_{\text{outlet}}^T / p_{\text{inlet}}^T$ is the total pressure ratio, and γ denotes adiabatic exponent. In the meantime, the design point flow of the optimized design is constrained so that its change does not exceed 2% of the reference design mass flow. The constraint is realized in the form of penalty function $p(x)$ (Song *et al.* 2016).

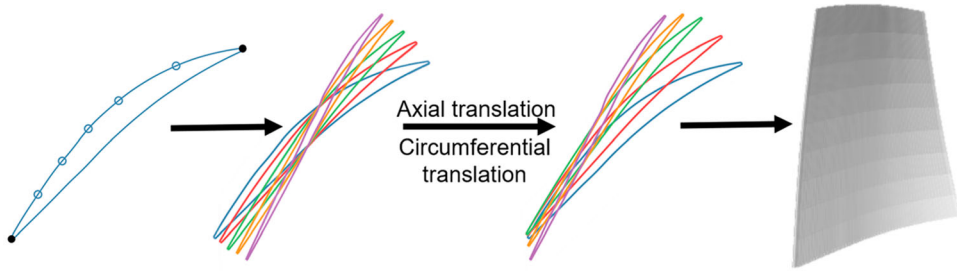


Figure 6. Sketch map of the 3-D parameterization method.

Table 4. Design conditions of the Rotor 37 blade.

Condition name	Value
Equivalent rotational speed (rpm)	17,188.7
Number of rotor blades	36
Rotor blade aspect ratio	1.19
Tip clearance gap (mm)	0.356
Inlet total temperature (K)	288.15
Inlet total pressure (Pa)	101,325

5.1.2. Numerical simulation model

The Rotor 37 blade is one of the rotors of the four-stage axial flow compressor Stage 37 with a high-pressure ratio. It was designed and tested by Reid and Moore in the NASA Glenn Center in the 1970s. The design parameters are taken as the inlet stage parameters of a typical aero-engine. Table 4 shows the relevant design parameters of Rotor 37, which remain the same as in the literature (Boretti 2010). To be in accordance with the literature, uniform total pressure and temperature are imposed at the inlet boundary, and an averaged static pressure is imposed at the outlet. The optimization is carried out with a constant outlet static pressure of 115,000 Pa, corresponding to a relative mass flow rate of 99% for the Rotor 37 blade.

For the above model, a grid with about 4×10^6 nodes is used for the calculation. The H-O-I topology is employed for the generation of the grid by using the commercial software NUMECA AutoGrid5™, and the mesh thickness of the first layer near the wall is set to be 3×10^{-6} m. In the calculation, the Spalart–Allmaras turbulence model is used with the adiabatic smooth walls condition. The Reynolds-averaged Navier–Stokes equations are solved by using the commercial software NUMECA FINE™/Turbo. With an Intel® i5-9400F@2.9 GHz CPU, the calculation time for a single sample is about 600 s. In the design condition, the efficiency, pressure ratio and mass flow of the reference design are 85.39%, 2.0395 and 20.62 kg/s, respectively. Figure 7 shows a comparison between the calculated results and the experimental results. It is easy to see that the calculated CFD results are in good agreement with the experimental results, which verifies the correctness of the CFD calculation method.

5.1.3. Analysis of results

These algorithms are used to optimize the Rotor 37 blade 5 times independently, and the total number of CFD calculations in each optimization is 1000. The optimization results are shown in Table 5. All five algorithms improve the efficiency of the blade, among which the optimization results of the DA-EGO algorithm are the best, which has great advantages in terms of the convergence efficiency and robustness compared with the other algorithms. After 1000 CFD calculations with the DA-EGO algorithm, the total efficiency of the Rotor 37 blade was improved by 1.65%. The variance of the results of five repeated operations of the DA-EGO, GA and DE algorithms is very small, indicating that these algorithms are less affected by the initial distribution of samples.

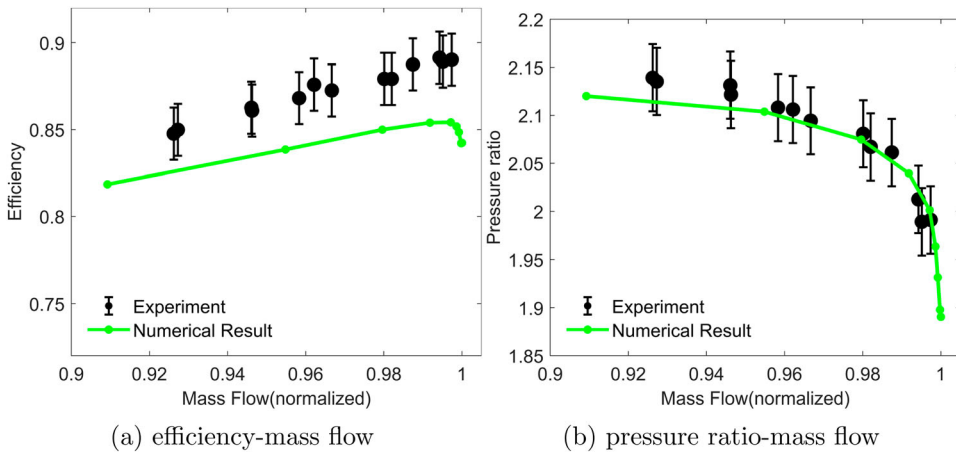


Figure 7. Comparison of feature curves of CFD simulation and experimental results. (a) Efficiency–mass flow and (b) Pressure ratio–mass flow.

Table 5. Optimization results from engineering test case 1 in detail.

Algorithm	Efficiency	Pressure ratio	Mass flow (kg/s)	Efficiency improvement (%)
Baseline	0.8539	2.040	20.62	–
DA-EGO	0.8707	2.055	20.96	1.65
GSGA	0.8696	2.051	20.89	1.57
IKAEA	0.8665	2.050	20.88	1.26
GA	0.8673	2.049	20.88	1.34
DE	0.8645	2.049	20.83	1.06

Figure 8 exhibits the suction surface’s limited flow for the baseline design and optimization results from the DA-EGO algorithm. This comparison reveals the presence of the ‘shock-wave–boundary-layer interference’ phenomenon on the baseline Rotor 37 suction surface, and severe flow separation near the trailing edge. Furthermore, back-flow is involved near the trailing tip edge. Comparing Figures 8(a) and 8(b), highly-optimized separation lines resulted in reduced areas of the separation and intensity of the involved back-flow, which is beneficial for the improvement of efficiency.

5.2. Engineering test case 2: multi-stage compressor

5.2.1. Problem description

In this section, a typical multistage axial flow compressor design task is used to demonstrate the efficiency of the DA-EGO algorithm in engineering tasks with more design variables. The geometry and mesh of the design object is shown in Figure 9. This multistage axial compressor consists of five rows of blades that contain one Inlet Guide Vane (IGV) and two compressor stages. In this case, the grid used in the optimization has about 2.3×10^7 nodes and is also generated by the software AutoGrid5. With the Intel i5-9400F@2.9 GHz CPU, the calculation time of a single sample of the multi-stage compressor is about 4200 s. Table 6 shows the relevant design parameters of the above multistage axial compressor.

Similar to the previous engineering test case, the parameterization method of each blade is shown in Figure 6. In each stage, whose rotor blade and stator are labelled R1/R2 and S1/S2, three section profiles of 0%, 50% and 100% span are selected from each blade. Considering that the flow in the rotor blade’s passage is more complex, five active control points are selected from each section profile’s suction curve in the rotor blade, and only three control points are selected in the stator’s suction. In addition to the three variables that control the bending and sweeping, finally, rotor blade R1/R2

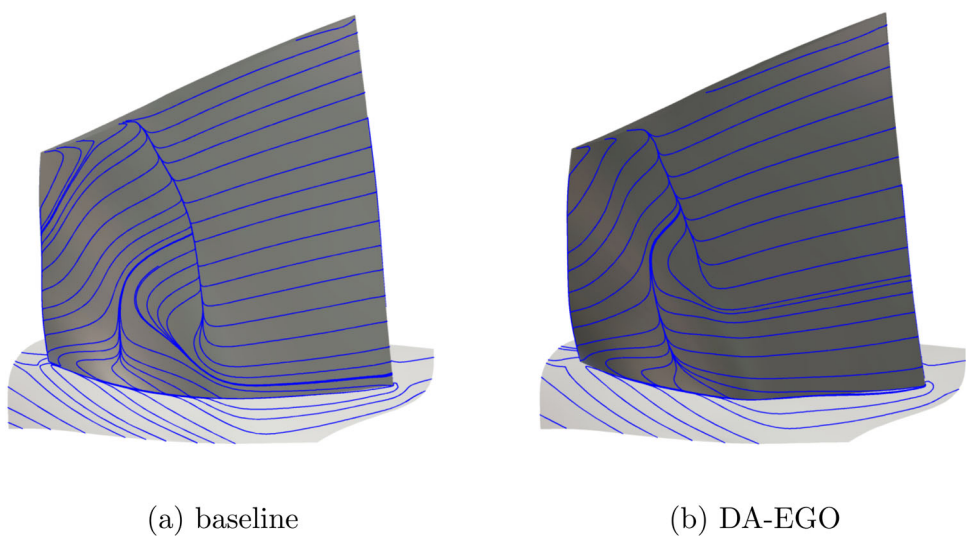


Figure 8. The limit streamlines the suction surface of the DA-EGO algorithms and the baseline design of Rotor 37 blade. (a) Baseline and (b) DA-EGO.

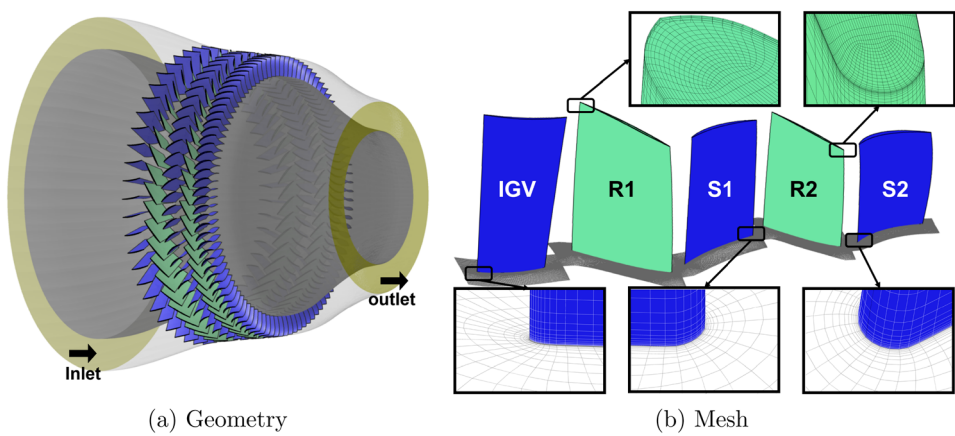


Figure 9. Geometric model and mesh of the multistage compressor. (a) Geometry and (b) Mesh.

Table 6. Design conditions of the multistage compressor.

Condition name	Value
Equivalent rotational speed (rpm)	8614.2
Number of rotor blades	51/47/67/57/97
Tip clearance gap (mm)	0.3
Inlet total temperature (K)	288.15
Inlet total pressure (Pa)	101,325
Design average outlet static pressure (Pa)	220,000
Design total pressure ratio	2.34

has 18 design variables and stator blade S1/S2 has 12 design variables. As the IGV blade remains the same in this optimization design, it has in total 60 design variables for the remaining four rows of blades.

Table 7. Optimization results from engineering test case 2 of the multistage axial compressor in detail.

Algorithm	Efficiency	Pressure ratio	Mass flow(kg/s)	Efficiency improvement
Baseline	88.072%	2.4604	53.6465	—
DA-EGO	89.142%	2.4644	53.9005	1.070%
GSGA	88.973%	2.4630	53.7780	0.901%
IKAEA	88.905%	2.4623	53.8325	0.833%
GA	88.804%	2.4634	53.8110	0.732%
DE	88.495%	2.4575	53.5025	0.423%

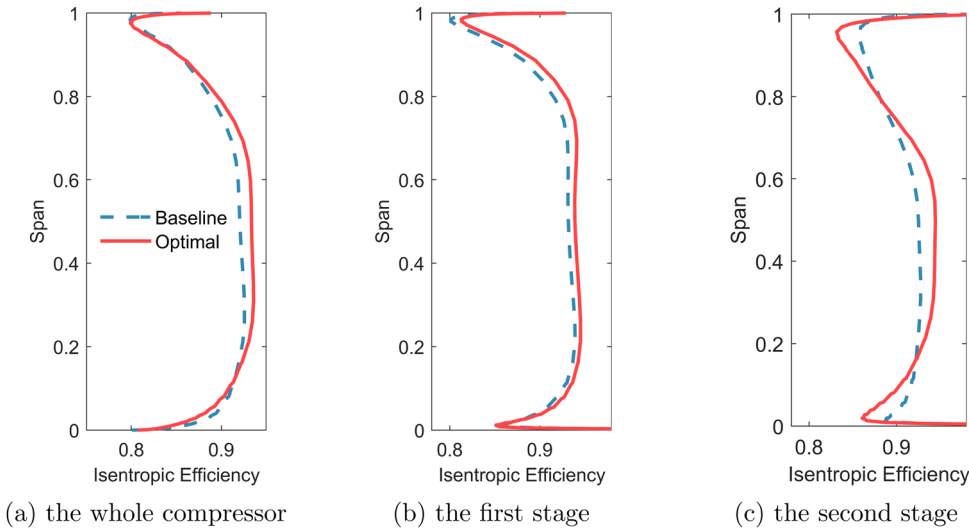


Figure 10. Total efficiency distribution along blade span. (a) Whole compressor. (b) First stage and (c) Second stage.

5.2.2. Results analysis

The DA-EGO algorithm and the other four compared algorithms are used to optimize the multistage axial compressor simultaneously, and the total number of CFD calculations in each optimization is also 1000. The optimization results are shown in Table 7. It is obvious that the DA-EGO algorithm is significantly better than all the other compared algorithms on the multistage compressor design optimization test. The proposed DA-EGO method improves the total efficiency of the multistage compressor by 1.07%.

In Figure 10, the distribution of efficiency along the span is shown. The total efficiency distributions of the whole compressor, the first stage and the second stage are shown in Figures 10(a), 10(b) and 10(c), respectively. Comparing the efficiency before and after optimizing by the DA-EGO algorithm: on the whole, the efficiency of the place higher than 90% span changed little, and in the middle span increased, obviously. While in the place near the endwall, the efficiency decreased slightly; Then, in the first stage, the efficiency increased by about 1% in all different span; And in the second stage, the efficiency at the height lower than 10% span and higher than 85% span decreased, while the efficiency improvement at the middle span was very obvious.

Figure 11 shows the distribution of the entropy function on the blade surface before and after optimization. It can be seen that the three regions with the highest entropy in the flow passage (labelled as regions A, B and C, respectively), all show significant decreases in their entropy values after optimization. Among them, the entropy in regions A and B decreases most obviously. The reason is that the change of blade profile weakens the shock waves in rotor R1's channel, which makes the entropy around the shock waves decrease. Further, the static pressure distribution diagram of the middle axis

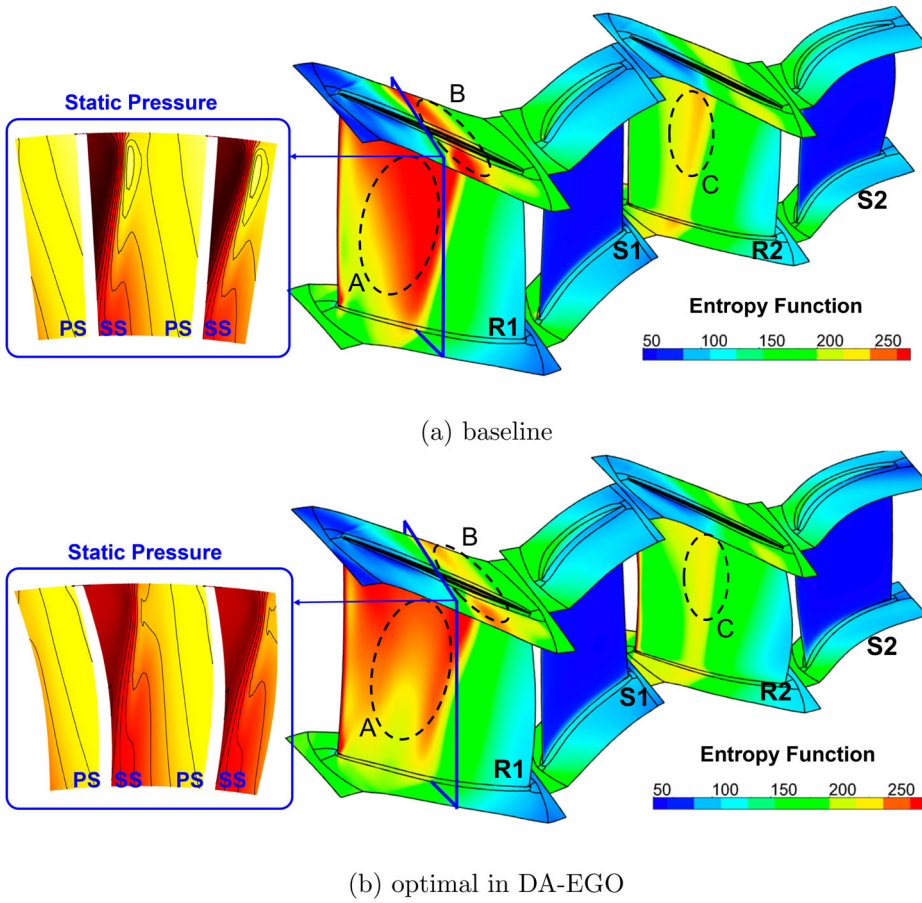


Figure 11. Comparison of entropy function contour of DA-EGO's optimization result and the baseline design in engineering case 2. (a) Baseline and (b) Optimal in DA-EGO.

section rotor R1's channel is drawn on the left side. After optimization, the uniformity of the circumferential static pressure distribution is greatly increased, which also proves the weakening of shock waves.

The empirical results of the engineering optimization design indicate the feasibility and practicality of the proposed DA-EGO algorithm in addressing large variable high-dimensional engineering optimization problems, with a marked improvement in efficiency as compared to other conventional methods. This conclusively establishes the efficacy of the DA-EGO algorithm. Hence, the effectiveness of the proposed DA-EGO algorithm has been demonstrated.

6. Conclusions

In the face of regularly occurring high-dimensional engineering design challenges in real-world scenarios, this study introduces a new algorithm, the Dynamic Aggregate Efficient Global Optimization (DA-EGO) algorithm, designed specifically for resolving high-dimension and expensive black-box (HEB) issues. The DA-EGO algorithm decomposes high-dimension problems into multiple simpler lower-dimension subproblems, and its core concept involves using a surrogate model to collect knowledge from individual subspaces that are then aggregated to develop a 'global' comprehension of the original design space. This aggregated knowledge forms the basis for creating the next cycle's

decomposition scheme of the DA-EGO algorithm. Additionally, the space reduction method is also used to enhance sub-task optimization efficiency further, while maintaining interaction analysis accuracy. Subsequently, this article provides thorough validation of the DA-EGO algorithm by comparing its performance against state-of-the-art optimization algorithms across a wide range of diverse test cases. These test cases include typical functions with dimensions from 30 to 90, a 28-dimensional design process of the Rotor 37 compressor blade, and a standard multi-stage compressor with 60-dimensions. These evaluations provide strong evidence for the validity and practical utility of the DA-EGO algorithm.

Currently, this study has specific limitations. The method's drawback lies in elevating the time complexity of the algorithm, leading to augmented consumption of computing resources by the algorithm. While this poses no hindrance to optimizing large resource problems, it diminishes the algorithm's performance in scenarios involving inexpensive samples. The primary computation for the sample occurs during the construction of the Kriging model, and this issue can be addressed using the enhanced Kriging method.

Disclosure statement

No potential conflict of interest was reported by the author(s).

Funding

This work was supported by the National Science and Technology Major Project [2019-II-0008-0028]; the Industry-University-Research Cooperation Project of the Aero Engine Corporation of China [HFZL2021CXY004]; and the High-Level Innovative and Entrepreneurial Talents Introduction Project of Qinchuangyuan [QCYRCXM-2022-210].

Data availability statement

The data that support the findings of this study are available from https://github.com/zhet1997/DA_EGO_publish.

References

- Boretti, A. 2010. "Experimental and Computational Analysis of a Transonic Compressor Rotor." In *Proceedings of the 17th Australasian Fluid Mechanics Conference*, 473–476. Red Hook, NY: Curran Associates. https://people.eng.unimelb.edu.au/imarusic/proceedings/17/016_Paper.pdf.
- Bouhellel, Mohamed Amine, Nathalie Bartoli, Rommel G. Regis, Abdelkader Otsmane, and Joseph Morlier. 2018. "Efficient Global Optimization for High-Dimensional Constrained Problems by Using the Kriging Models Combined with the Partial Least Squares Method." *Engineering Optimization* 50 (12): 2038–2053. <https://doi.org/10.1080/0305215X.2017.1419344>.
- Cai, Xiwen, Haobo Qiu, Liang Gao, Chen Jiang, and Xinyu Shao. 2019. "An Efficient Surrogate-Assisted Particle Swarm Optimization Algorithm for High-Dimensional Expensive Problems." *Knowledge-Based Systems* 184: 104901. <https://doi.org/10.1016/j.knsys.2019.104901>.
- Chen, Wenxiang, Thomas Weise, Zhenyu Yang, and Ke Tang. 2010. "Large-Scale Global Optimization Using Cooperative Coevolution with Variable Interaction Learning." In *Proceedings of the International Conference on Parallel Problem Solving from Nature (PPSN XI)*, edited by Robert Schaefer, Carlos Cotta, Joanna Kołodziej and Günter Rudolph, 300–309. Berlin: Springer.
- Derrac, Joaquín, Salvador García, Daniel Molina, and Francisco Herrera. 2011. "A Practical Tutorial on the Use of Non-parametric Statistical Tests As a Methodology for Comparing Evolutionary and Swarm Intelligence Algorithms." *Swarm and Evolutionary Computation* 1 (1): 3–18. <https://doi.org/10.1016/j.swevo.2011.02.002>.
- Forrester, Alexander I. J., Neil W. Bressloff, and Andy J. Keane. 2006. "Optimization Using Surrogate Models and Partially Converged Computational Fluid Dynamics Simulations." *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences* 462 (2071): 2177–2204. <https://doi.org/10.1098/rspa.2006.1679>.
- Hajikolaie, Kambiz Haji, and G. Gary Wang. 2014. "High Dimensional Model Representation with Principal Component Analysis." *Journal of Mechanical Design* 136 (1): 011003. <https://doi.org/10.1115/1.4025491>.
- Jiang, Puyu, Yuansheng Cheng, and Jun Liu. 2022. "Cooperative Bayesian Optimization with Hybrid Grouping Strategy and Sample Transfer for Expensive Large-Scale Black-Box Problems." *Knowledge-Based Systems* 254: 109633. <https://doi.org/10.1016/j.knsys.2022.109633>.
- Jones, Donald R., and Matthias Schonlau. 1998. "Efficient Global Optimization of Expensive Black-Box Functions." *Journal of Global Optimization* 13 (4): 455–492. <https://doi.org/10.1023/A:1008306431147>.

- Kang, Kyeonghwan, and Ikjin Lee. 2020. "Efficient High-Dimensional Metamodeling Strategy Using Recursive Decomposition Coupled with Sequential Sampling Method." *Structural and Multidisciplinary Optimization* 63:375–390. <https://doi.org/10.1007/s00158-020-02705-1>.
- Li, Genghui, Lindong Xie, Zhenkun Wang, Huajun Wang, and Maoguo Gong. 2023. "Evolutionary Algorithm with Individual-Distribution Search Strategy and Regression-Classification Surrogates for Expensive Optimization." *Information Sciences* 634:423–442. <https://doi.org/10.1016/j.ins.2023.03.101>.
- Liu, Ruochen, Jianxia Li, Jing Fan, and Licheng Jiao. 2018. "A Dynamic Multiple Populations Particle Swarm Optimization Algorithm Based on Decomposition and Prediction." *Applied Soft Computing* 73:434–459. <https://doi.org/10.1016/j.asoc.2018.08.015>.
- Marelli, Stefano, and Bruno Sudret. 2014. "UQLab: A Framework for Uncertainty Quantification in Matlab." In *Proceedings of the 2nd International Conference on Vulnerability, Risk Analysis and Management (ICVRAM2014) Vulnerability, Uncertainty, and Risk*, 2554–2563. Reston, VA: ASCE (American Society of Civil Engineers). <https://doi.org/10.1061/9780784413609.257>.
- Mei, Yi, Mohammad Nabi Omidvar, Xiaodong Li, and Xin Yao. 2016. "A Competitive Divide-and-Conquer Algorithm for Unconstrained Large-Scale Black-Box Optimization." *ACM Transactions on Mathematical Software* 42 (2): 1–24. <https://doi.org/10.1145/2791291>.
- Meselhi, Mohamed, Ruhul Sarker, Daryl Essam, and Saber Elsayed. 2022. "A Decomposition Approach for Large-Scale Non-Separable Optimization Problems." *Applied Soft Computing* 115:108168. <https://doi.org/10.1016/j.asoc.2021.108168>.
- Mitchell, Melanie. 1998. *An Introduction to Genetic Algorithms*. Cambridge, MA: MIT Press. <https://doi.org/10.7551/mitpress/3927.001.0001>.
- Mohamed, Ali Wagdy, Anas A. Hadi, and Ali Khater Mohamed. 2020. "Gaining-Sharing Knowledge Based Algorithm for Solving Optimization Problems: A Novel Nature-Inspired Algorithm." *International Journal of Machine Learning and Cybernetics* 11 (7): 1501–1529. <https://doi.org/10.1007/s13042-019-01053-x>.
- Okulewicz, Michał, Mateusz Zaborski, and Jacek Mańdziuk. 2022. "Self-Adapting Particle Swarm Optimization for Continuous Black Box Optimization." *Applied Soft Computing* 131:109722. <https://doi.org/10.1016/j.asoc.2022.109722>.
- Omidvar, Mohammad Nabi, Xiaodong Li, Yi Mei, and Xin Yao. 2014. "Cooperative Co-Evolution with Differential Grouping for Large Scale Optimization." *IEEE Transactions on Evolutionary Computation* 18 (3): 378–393. <https://doi.org/10.1109/TEVC.4235>.
- Omidvar, Mohammad Nabi, Xiaodong Li, and Xin Yao. 2010. "Cooperative Co-Evolution with Delta Grouping for Large Scale Non-Separable Function Optimization." In *Proceedings of the IEEE Congress on Evolutionary Computation*, 1–8. Piscataway, NJ: IEEE.
- Potter, Mitchell A., and Kenneth A. De Jong. 1994. "A Cooperative Coevolutionary Approach to Function Optimization." In *Proceedings of the International Conference on Evolutionary Computation. The Third Conference on Parallel Problem Solving from Nature (PPSN III)*, edited by Gerhard Goos, Juris Hartmanis, Jan Leeuwen, Yuval Davidor, Hans-Paul Schwefel, and Reinhard Männer, 249–257. Berlin: Springer-Verlag. <https://core.ac.uk/display/24351662>.
- Qian, Peter Z. G., and C. F. Jeff Wu. 2008. "Bayesian Hierarchical Modeling for Integrating Low-Accuracy and High-Accuracy Experiments." *Technometrics* 50 (2): 192–204. <https://doi.org/10.1198/004017008000000082>.
- Shan, Songqing, and G. Gary Wang. 2010. "Survey of Modeling and Optimization Strategies to Solve High-Dimensional Design Problems with Computationally-Expensive Black-Box Functions." *Structural and Multidisciplinary Optimization* 41 (2): 219–241. <https://doi.org/10.1007/s00158-009-0420-2>.
- Song, Liming, Zhendong Guo, Jun Li, and Zhenping Feng. 2016. "Research on Metamodel-Based Global Design Optimization and Data Mining Methods." *Journal of Engineering for Gas Turbines and Power* 138 (9): 092604. <https://doi.org/10.1115/1.4032653>.
- Storn, Rainer, and Kenneth Price. 1997. "Differential Evolution—A Simple and Efficient Heuristic for Global Optimization Over Continuous Spaces." *Journal of Global Optimization* 11 (4): 341–359. <https://doi.org/10.1023/A:1008202821328>.
- Suder, Kenneth L. 1996. "Experimental Investigation of the Flow Field in a Transonic, Axial Flow Compressor with Respect to the Development of Blockage and Loss." PhD diss. Cleveland, OH: NASA. <https://ntrs.nasa.gov/citations/19970001675>.
- Sudret, Bruno. 2008. "Global Sensitivity Analysis Using Polynomial Chaos Expansions." *Reliability Engineering & System Safety* 93 (7): 964–979. <https://doi.org/10.1016/j.res.2007.04.002>.
- Sun, Yuan, Michael Kirley, and Saman K. Halgamuge. 2018. "A Recursive Decomposition Method for Large Scale Continuous Optimization." *IEEE Transactions on Evolutionary Computation* 22 (5): 647–661. <https://doi.org/10.1109/TEVC.2017.2778089>.
- Tang, Ke, Xiaodong Li, Ponnuthurai Suganthan, Zhenyu Yang, and Thomas Weise. 2009. *Benchmark Functions for the CEC'2008 Special Session and Competition on Large Scale Global Optimization*. Vol. 1. Technical Report, University of Science and Technology of China. https://www.researchgate.net/publication/228932005_Benchmark_functions_for_the_CEC'2008_special_session_and_competition_on_large_scale_global_optimization.

- Tang, Zhili, Liang Xu, and Shaojun Luo. 2022. "Adaptive Dynamic Surrogate-Assisted Evolutionary Computation for High-Fidelity Optimization in Engineering." *Applied Soft Computing* 127:109333. <https://doi.org/10.1016/j.asoc.2022.109333>.
- Wang, Kai, Zhonghua Han, Keshi Zhang, and Wenping Song. 2023. "Efficient Global Aerodynamic Shape Optimization of a Full Aircraft Configuration Considering Trimming." *Aerospace* 10 (8): 734. <https://doi.org/10.3390/aerospace10080734>.
- Wang, Jian, Seokbeop Kwon, and Byonghyo Shim. 2012. "Generalized Orthogonal Matching Pursuit." *IEEE Transactions on Signal Processing* 60 (12): 6202–6216. <https://doi.org/10.1109/TSP.2012.2218810>.
- Wang, Qineng, Liming Song, Yun Chen, Guangjian Ma, Zhendong Guo, and Jun Li. 2023. "KT-EGO: A Knowledge Transfer Assisted Efficient Global Optimization Algorithm for Solving High-Dimensional Expensive Black-Box Problems." *Engineering Optimization* 55 (12): 2015–2033. <https://doi.org/10.1080/0305215X.2022.2139374>.
- Wiener, Norbert. 1938. "The Homogeneous Chaos." *American Journal of Mathematics* 60 (4): 897–936. <https://doi.org/10.2307/2371268>.
- Xie, Lindong, Genghui Li, Zhenkun Wang, Laizhong Cui, and Maoguo Gong. 2023. "Surrogate-Assisted Evolutionary Algorithm with Model and Infill Criterion Auto-Configuration." *IEEE Transactions on Evolutionary Computation*. Advance online publication. <https://doi.org/10.1109/TEVC.2023.3291614>.
- Xiu, Dongbin. 2010. *Numerical Methods for Stochastic Computations: A Spectral Method Approach*. Princeton, NJ: Princeton University Press.
- Xu, Shengguan, and Hongquan Chen. 2018. "Nash Game Based Efficient Global Optimization for Large-Scale Design Problems." *Journal of Global Optimization* 71 (2): 361–381. <https://doi.org/10.1007/s10898-018-0608-3>.
- Yang, Zhenyu, Ke Tang, and Xin Yao. 2008. "Large Scale Evolutionary Optimization Using Cooperative Coevolution." *Information Sciences* 178 (15): 2985–2999. <https://doi.org/10.1016/j.ins.2008.02.017>.
- Yang, Peng, Ke Tang, and Xin Yao. 2018. "Turning High-Dimensional Optimization Into Computationally Expensive Optimization." *IEEE Transactions on Evolutionary Computation* 22 (1): 143–156. <https://doi.org/10.1109/TEVC.2017.2672689>.
- Zhan, Dawei, and Huanlai Xing. 2021. "A Fast Kriging-Assisted Evolutionary Algorithm Based on Incremental Learning." *IEEE Transactions on Neural Networks* 25 (5): 941–955. <https://doi.org/10.1109/TEVC.2021.3067015>.
- Zhang, Ning, Peng Wang, Huachao Dong, and Tianbo Li. 2020. "Shape Optimization for Blended-Wing-Body Underwater Glider Using An Advanced Multi-Surrogate-Based High-Dimensional Model Representation Method." *Engineering Optimization* 52 (12): 2080–2099. <https://doi.org/10.1080/0305215X.2019.1694674>.

Appendix

Table A1. The optimization results of eight algorithms on 30-D benchmark functions.

Function	Algorithm	Best	Worst	Mean	Std	Wilcoxon
F1	DA-EGO	2.226E−03	4.311E−02	1.730E−02	1.263E−02	N/A
	RG-EGO	1.170E−01	6.607E−01	3.295E−01	1.591E−01	+
	DE	7.722E+05	2.090E+06	1.336E+06	3.541E+05	+
	GA	2.339E+05	5.043E+06	1.814E+06	1.289E+06	+
	GSK	1.758E+05	1.470E+06	4.590E+05	2.633E+05	+
	Nash-EGO	1.359E+02	3.958E+02	2.442E+02	1.032E+02	+
	IKAEA	3.704E+04	2.456E+05	8.105E+04	4.950E+04	+
	GSGA	1.392E+05	5.068E+05	2.641E+05	1.005E+05	+
F2	DA-EGO	2.685E+01	5.859E+01	4.653E+01	8.412E+00	N/A
	RG-EGO	4.730E+01	8.265E+01	6.328E+01	1.007E+01	+
	DE	2.651E+02	3.493E+02	3.010E+02	2.188E+01	+
	GA	8.565E+01	1.730E+02	1.304E+02	2.604E+01	+
	GSK	2.226E+02	2.991E+02	2.622E+02	2.271E+01	+
	Nash-EGO	6.146E+01	6.146E+01	6.146E+01	7.290E−15	+
	IKAEA	2.058E+02	2.763E+02	2.501E+02	2.048E+01	+
	GSGA	6.408E+01	1.458E+02	9.642E+01	2.111E+01	+
F3	DA-EGO	1.234E+00	2.126E+00	1.522E+00	1.967E−01	N/A
	RG-EGO	1.896E+00	4.016E+00	3.152E+00	6.509E−01	+
	DE	1.659E+01	1.894E+01	1.816E+01	5.639E−01	+
	GA	1.013E+01	1.605E+01	1.341E+01	2.257E+00	+
	GSK	1.157E+01	1.542E+01	1.348E+01	1.023E+00	+
	Nash-EGO	1.034E+01	1.106E+01	1.092E+01	2.953E−01	+
	IKAEA	7.189E−01	1.904E+01	8.861E+00	4.752E+00	+
	GSGA	7.281E−01	4.747E+00	2.834E+00	1.197E+00	+
F4	DA-EGO	4.114E+03	2.843E+05	7.672E+04	8.002E+04	N/A
	RG-EGO	9.764E+04	3.615E+06	1.285E+06	1.302E+06	+
	DE	3.557E+06	7.844E+06	5.463E+06	1.130E+06	+
	GA	3.842E+05	3.715E+06	1.670E+06	9.448E+05	+
	GSK	9.913E+05	2.828E+06	1.667E+06	4.954E+05	+
	Nash-EGO	1.394E+06	1.450E+06	1.422E+06	2.197E+04	+
	IKAEA	1.052E+05	5.353E+05	2.387E+05	1.053E+05	+
	GSGA	2.373E+05	9.421E+07	5.194E+06	2.095E+07	+
F5	DA-EGO	4.203E+01	9.876E+01	6.992E+01	1.535E+01	N/A
	RG-EGO	8.555E+01	1.495E+02	1.135E+02	2.291E+01	+
	DE	2.707E+02	3.736E+02	3.155E+02	2.574E+01	+
	GA	1.167E+02	1.950E+02	1.586E+02	2.357E+01	+
	GSK	2.246E+02	2.912E+02	2.638E+02	1.660E+01	+
	Nash-EGO	1.282E+02	1.649E+02	1.512E+02	1.319E+01	+
	IKAEA	2.340E+02	3.062E+02	2.596E+02	1.659E+01	+
	GSGA	5.939E+01	1.320E+02	9.909E+01	1.874E+01	+
F6	DA-EGO	7.191E+00	4.419E+01	1.534E+01	9.795E+00	N/A
	RG-EGO	1.179E+01	4.094E+01	3.094E+01	8.138E+00	+
	DE	5.568E+01	7.117E+01	6.460E+01	3.402E+00	+
	GA	3.629E+01	6.150E+01	4.813E+01	8.936E+00	+
	GSK	4.550E+01	6.118E+01	5.400E+01	3.952E+00	+
	Nash-EGO	5.179E+01	5.996E+01	5.493E+01	3.127E+00	+
	IKAEA	2.415E+01	6.870E+01	4.442E+01	1.026E+01	+
	GSGA	2.700E+01	6.235E+01	4.370E+01	1.087E+01	+
F7	DA-EGO	1.906E+02	3.646E+03	3.178E+02	1.963E+03	N/A
	RG-EGO	5.451E+03	1.623E+04	1.583E+03	3.076E+03	+
	DE	2.394E+05	9.609E+05	5.187E+05	1.930E+05	+
	GA	7.232E+03	2.942E+04	1.444E+04	8.182E+03	+
	GSK	1.217E+04	1.136E+05	4.437E+04	2.838E+04	+
	Nash-EGO	5.506E+03	1.944E+04	6.241E+03	3.819E+03	+
	IKAEA	6.009E+02	2.276E+03	1.107E+03	5.688E+02	≈
	GSGA	1.371E+02	2.615E+03	3.160E+02	1.178E+03	≈

Note. 'Std' represents standard deviation, and 'Wilcoxon' represents the Wilcoxon signed rank test (Derrac *et al.* 2011); the symbols '+', '−' and '≈' indicate that the DA-EGO algorithm is significantly better than, significantly worse than or comparable with the compared algorithm, respectively; the optimal mean result of each function is marked in boldface type.

Table A2. The optimization results of seven algorithms on 60-D benchmark functions.

Function	Algorithm	Best	Worst	Mean	Std	Wilcoxon
F1	DA-EGO	1.658E−01	1.109E+00	4.646E−01	2.419E−01	N/A
	RG-EGO	6.170E−01	5.215E+00	1.849E+00	1.252E+00	+
	DE	5.749E+06	1.293E+07	1.017E+07	1.741E+06	+
	GA	2.912E+06	1.165E+07	6.508E+06	2.431E+06	+
	GSK	1.985E+06	5.321E+06	3.867E+06	9.036E+05	+
	IKAEA	2.219E+05	2.002E+06	6.489E+05	4.406E+05	+
	GSGA	9.081E+05	3.198E+06	1.884E+06	6.371E+05	+
F2	DA-EGO	1.078E+02	1.853E+02	1.559E+02	1.667E+01	N/A
	RG-EGO	1.793E+02	2.277E+02	2.098E+02	1.487E+01	+
	DE	6.428E+02	8.641E+02	7.802E+02	5.311E+01	+
	GA	3.668E+02	5.471E+02	4.680E+02	5.204E+01	+
	GSK	5.730E+02	6.953E+02	6.484E+02	2.756E+01	+
	IKAEA	3.347E+02	6.186E+02	5.304E+02	5.688E+01	+
	GSGA	1.643E+02	4.533E+02	3.051E+02	8.493E+01	+
F3	DA-EGO	3.658E+00	1.153E+01	6.218E+00	1.715E+00	N/A
	RG-EGO	9.216E+00	1.575E+01	1.271E+01	1.985E+00	+
	DE	1.993E+01	2.099E+01	2.057E+01	2.470E−01	+
	GA	1.585E+01	1.950E+01	1.799E+01	9.458E−01	+
	GSK	1.613E+01	1.857E+01	1.761E+01	6.819E−01	+
	IKAEA	3.693E+00	1.984E+01	9.455E+00	4.564E+00	+
	GSGA	3.939E+00	2.000E+01	8.008E+00	4.804E+00	≈
F4	DA-EGO	1.075E+05	1.348E+06	6.832E+05	3.276E+05	N/A
	RG-EGO	2.587E+07	4.843E+07	3.709E+07	7.014E+06	+
	DE	2.065E+07	4.580E+07	3.528E+07	6.510E+06	+
	GA	5.075E+06	2.030E+07	1.179E+07	4.384E+06	+
	GSK	6.776E+06	1.423E+07	9.155E+06	2.018E+06	+
	IKAEA	6.459E+05	2.122E+06	1.232E+06	4.474E+05	+
	GSGA	9.329E+05	6.320E+06	3.028E+06	1.560E+06	+
F5	DA-EGO	1.678E+02	2.762E+02	2.181E+02	2.816E+01	N/A
	RG-EGO	2.632E+02	4.174E+02	3.494E+02	4.988E+01	+
	DE	7.363E+02	9.297E+02	8.419E+02	4.457E+01	+
	GA	4.312E+02	6.199E+02	5.115E+02	5.680E+01	+
	GSK	5.739E+02	7.008E+02	6.529E+02	3.520E+01	+
	IKAEA	5.002E+02	6.510E+02	5.914E+02	3.522E+01	+
	GSGA	1.786E+02	5.540E+02	3.190E+02	9.178E+01	+
F6	DA-EGO	4.046E+01	8.948E+01	6.336E+01	1.436E+01	N/A
	RG-EGO	6.543E+01	1.066E+02	8.968E+01	1.231E+01	+
	DE	1.225E+02	1.321E+02	1.279E+02	2.607E+00	+
	GA	9.156E+01	1.217E+02	1.066E+02	8.609E+00	+
	GSK	9.602E+01	1.200E+02	1.076E+02	6.172E+00	+
	IKAEA	6.397E+01	9.959E+01	8.314E+01	1.093E+01	+
	GSGA	6.781E+01	1.136E+02	9.316E+01	1.179E+01	+
F7	DA-EGO	5.861E+03	2.972E+04	1.539E+04	5.032E+03	N/A
	RG-EGO	6.643E+04	1.263E+05	6.590E+04	1.599E+04	+
	DE	3.474E+06	7.241E+06	5.067E+06	9.756E+05	+
	GA	4.631E+05	1.886E+06	9.006E+05	3.965E+05	+
	GSK	2.757E+05	9.893E+05	6.517E+05	2.124E+05	+
	IKAEA	1.401E+04	2.108E+05	6.207E+04	7.425E+04	+
	GSGA	2.127E+03	2.614E+04	9.931E+03	3.571E+03	−

Note. 'Std' represents standard deviation, and 'Wilcoxon' represents the Wilcoxon signed rank test (Derrac *et al.* 2011); the symbols '+', '−' and '≈' indicate that the DA-EGO algorithm is significantly better than, significantly worse than or comparable with the compared algorithm, respectively; the optimal mean result of each function is marked in boldface type.

Table A3. The average result of computation time and optimal value of three benchmark functions.

Function	Algorithm	Average computation time	Average optimal value
Elliptic-30D	DA-EGO	35 min	1.73E−02
	RG-EGO	36 min	3.30E−01
	DE	0.741 s	1.34E+06
	GA	3.2 s	1.81E+06
	GSK	0.124 s	4.59E+05
	Nash-EGO	46 min	2.44E+02
	IKAEA	68 s	8.11E+04
	GSGA	26 min	2.64E+05
Elliptic-60D	DA-EGO	36 min	4.65E−01
	RG-EGO	33 min	1.85E+00
	DE	0.919 s	1.02E+07
	GA	3.31 s	6.51E+06
	GSK	0.139 s	3.87E+06
	IKAEA	101 s	6.49E+05
	GSGA	30 min	1.88E+06
Elliptic-90D	DA-EGO	40 min	2.10E+00
	RG-EGO	39 min	1.13E+01
	DE	1.264 s	2.89E+07
	GA	3.65 s	2.19E+07
	GSK	0.194 s	9.85E+06
	IKAEA	149 s	1.54E+06
	GSGA	37 min	6.84E+06

Table A4. The optimization results of seven algorithms on 90-D benchmark functions.

Func	Algo	Best	Worst	Mean	Std	Wilcoxon
F1	DA-EGO	9.531E−01	4.996E+00	2.096E+00	1.044E+00	N/A
	RG-EGO	3.997E+00	5.521E+01	1.128E+01	9.817E+00	+
	DE	2.184E+07	3.440E+07	2.886E+07	4.476E+06	+
	GA	1.490E+07	2.899E+07	2.191E+07	4.544E+06	+
	GSK	5.962E+06	1.461E+07	9.845E+06	2.433E+06	+
	IKAEA	8.137E+05	1.954E+06	1.535E+06	4.015E+05	+
	GSGA	4.987E+06	9.107E+06	6.841E+06	1.429E+06	+
F2	DA-EGO	2.530E+02	3.298E+02	2.974E+02	2.054E+01	N/A
	RG-EGO	3.408E+02	4.434E+02	3.962E+02	2.815E+01	+
	DE	1.284E+03	1.498E+03	1.391E+03	6.635E+01	+
	GA	8.799E+02	1.094E+03	9.776E+02	6.262E+01	+
	GSK	9.438E+02	1.133E+03	1.063E+03	3.916E+01	+
	IKAEA	7.128E+02	9.533E+02	8.487E+02	7.050E+01	+
	GSGA	6.935E+02	8.783E+02	7.793E+02	5.303E+01	+
F3	DA-EGO	9.354E+00	1.289E+01	1.108E+01	8.810E−01	N/A
	RG-EGO	1.092E+01	1.871E+01	1.532E+01	2.183E+00	+
	DE	2.065E+01	2.106E+01	2.094E+01	1.234E−01	+
	GA	1.899E+01	1.993E+01	1.954E+01	3.011E−01	+
	GSK	1.836E+01	1.971E+01	1.908E+01	3.283E−01	+
	IKAEA	9.623E+00	1.944E+01	1.290E+01	2.833E+00	≈
	GSGA	1.226E+01	2.035E+01	1.614E+01	3.021E+00	+
F4	DA-EGO	6.789E+05	6.384E+06	2.657E+06	1.525E+06	N/A
	RG-EGO	2.093E+06	1.427E+08	6.297E+07	5.888E+07	+
	DE	8.382E+07	1.023E+08	8.726E+07	6.402E+06	+
	GA	1.531E+07	5.085E+07	3.404E+07	9.625E+06	+
	GSK	1.018E+07	3.066E+07	1.678E+07	4.453E+06	+
	IKAEA	1.970E+06	4.320E+06	2.854E+06	7.711E+05	≈
	GSGA	7.175E+06	1.533E+07	1.070E+07	2.634E+06	+
F5	DA-EGO	3.599E+02	5.180E+02	4.343E+02	4.069E+01	N/A
	RG-EGO	4.464E+02	6.829E+02	5.517E+02	5.813E+01	+
	DE	1.378E+03	1.554E+03	1.454E+03	6.028E+01	+
	GA	8.977E+02	1.060E+03	9.928E+02	4.550E+01	+
	GSK	9.815E+02	1.132E+03	1.058E+03	4.032E+01	+
	IKAEA	8.332E+02	1.040E+03	9.386E+02	6.901E+01	+
	GSGA	6.992E+02	8.903E+02	8.353E+02	6.896E+01	+
F6	DA-EGO	9.898E+01	1.584E+02	1.286E+02	1.479E+01	N/A
	RG-EGO	1.035E+02	1.708E+02	1.499E+02	1.823E+01	+
	DE	1.833E+02	1.976E+02	1.920E+02	3.873E+00	+
	GA	1.620E+02	1.787E+02	1.686E+02	6.023E+00	+
	GSK	1.456E+02	1.712E+02	1.615E+02	7.193E+00	+
	IKAEA	9.574E+01	1.513E+02	1.242E+02	1.844E+01	≈
	GSGA	1.316E+02	1.611E+02	1.481E+02	7.939E+00	+
F7	DA-EGO	5.106E+04	8.361E+04	1.418E+05	1.616E+04	N/A
	RG-EGO	5.941E+05	9.661E+05	3.214E+05	7.055E+04	+
	DE	8.981E+06	1.646E+07	1.292E+07	2.854E+06	+
	GA	2.852E+06	5.586E+06	4.298E+06	8.603E+05	+
	GSK	1.152E+06	2.835E+06	2.048E+06	3.584E+05	+
	IKAEA	6.421E+04	2.155E+05	1.044E+05	4.341E+04	≈
	GSGA	5.116E+04	1.150E+05	9.871E+04	1.920E+04	−

Note. 'Std' represents standard deviation, and 'Wilcoxon' represents the Wilcoxon signed rank test (Derrac *et al.* 2011); the symbols '+', '−' and '≈' indicate that the DA-EGO algorithm is significantly better than, significantly worse than or comparable with the compared algorithm, respectively; the optimal mean result of each function is marked in boldface type.